

Multimode Rabi dynamics: finite models, experiments and patch analysis

Documentation of the `rabi-truncation` repository

28 September 2026

Abstract

The repository evolves pure states with one shared engine, enumerated photon occupation bases and sparse Hamiltonians. This document specifies the finite models actually solved, derives their preparation and measurement formulas, explains the common-space embedding used for fidelities, and documents the Python and notebook interfaces. The comparison with the student implementation separates unitary changes of convention, changes of the finite physical model, and corrections to numerical comparisons. Numbered source listings are followed by comments tied to their current lines. The common `full` space used for overlaps is implicit; choosing a separately propagated comparison trajectory does not require propagating or allocating that ambient space.

Contents

1	Finite model, state preparation and calculated quantities	1
1.1	Periodic grid, parameter controls and photon spaces	1
1.2	Preparation, projected Hamiltonian and evolution	4
1.3	Observables, exact fidelities and physical interpretation	7
2	Notebook control, resources and source-line commentary	17
2.1	Shared configuration and callable Python experiments	17
2.2	Correspondence between controls and resource costs	19
2.3	Numbered code and comments on individual operations	21
3	Comparison with the student implementation and validation	42
3.1	Exact convention conversions and finite-model changes	42
3.2	Mathematical verification and limits of the conclusions	45

1 Finite model, state preparation and calculated quantities

1.1 Periodic grid, parameter controls and photon spaces

Notation and units. Throughout, $\hbar = c = 1$. Length and time have inverse-energy units. The TLS energy origin is its ground state: its free Hamiltonian is $\omega_0 |e\rangle \langle e|$, rather than $\omega_0 \sigma_z/2$. These differ by a scalar, which matters when reporting an absolute energy. The symbols used below are:

Symbol	Meaning in the implementation
L	Periodic box length, supplied in <code>param_atom</code> .
k_m, ω_m	Signed momentum and nonnegative frequency $\omega_m = k_m $.
M	Number of retained modes; after selection this means the selected count, unless explicitly described as the base-grid count.
N, B, d	<code>n_max</code> , number of photon configurations, and joint ket dimension $d = 2B$.
$\mathbf{n}, \nu$	Occupation tuple and total photon number $\nu = \sum_m n_m$.
D, f_m, u_m	Physical coupling, chosen profile and complex annihilation coefficient, defined in equation (30).
k_0, σ_k, x_0	Packet momentum centre, width convention and initial Fourier position parameter.
x_{tls}	Parameter in the imposed annihilation phase; its plotted interaction coordinate is derived in equation (47).
T, dt, N_t	Final time, nominal output spacing and number of output times, including both endpoints.

Grid and the two controls. For a periodic box of length $L > 0$,

$$\varphi_j(x) = \frac{e^{ik_j x}}{\sqrt{L}}, \quad \Delta k = \frac{2\pi}{L}, \quad k_j = j\Delta k, \quad j \in \mathbb{Z}, \quad \omega_j = |k_j|. \quad (1)$$

Indeed, $\varphi_j(x + L) = \varphi_j(x)$ and $\int_{x_*}^{x_*+L} \varphi_j^* \varphi_\ell dx = \delta_{j\ell}$. Signed momenta enter spatial phases; their absolute values enter the dispersion and square-root profile. Modes are sorted by signed momentum. There is no automatic deletion of the zero mode.

When `CTRL_M_EXPLICIT=True`, the inputs are L and a positive odd integer M . Setting $J = (M - 1)/2$ gives

$$\mathcal{K}_M = \{j\Delta k : -J \leq j \leq J\}, \quad |\mathcal{K}_M| = M, \quad k_{\text{IR},\text{eff}} = 0, \quad k_{\text{UV},\text{eff}} = \frac{M-1}{2} \Delta k. \quad (2)$$

The cutoff dictionary is inactive. An even M raises **ValueError**; it is never rounded, decremented or made asymmetric. $M = 1$ gives $\mathcal{K} = \{0\}$, not a resonant single-mode cavity at ω_0 . Booleans and floating-point values, including 5.0, are rejected as mode-count inputs; Python and NumPy integer types are accepted.

When `CTRL_M_EXPLICIT=False`, the inputs are L and finite radial cutoffs $0 \leq k_{\text{IR}} \leq k_{\text{UV}}$:

$$\mathcal{K}_{\text{IR},\text{UV}} = \{j\Delta k : k_{\text{IR}} \leq |j|\Delta k \leq k_{\text{UV}}\}. \quad (3)$$

For strictly positive indices, in exact arithmetic,

$$j_{\min} = \max\left(1, \left\lceil \frac{k_{\text{IR}}}{\Delta k} \right\rceil\right), \quad j_{\max} = \left\lfloor \frac{k_{\text{UV}}}{\Delta k} \right\rfloor, \quad (4)$$

$$M_{\text{eff}} = 2 \max(0, j_{\max} - j_{\min} + 1) + \mathbf{1}_{k_{\text{IR}}=0}. \quad (5)$$

The code subtracts 10^{-12} before the ceiling and adds 10^{-12} before the floor, protecting numerically boundary-aligned points. This tolerance is on the dimensionless cutoff-to-spacing ratios. Zero is included if and only if the input IR equals zero exactly. A positive IR excludes it through the band definition even when that IR is very small. An empty band is an error. The resulting count can be even when zero is excluded; the odd- M condition belongs only to the explicit-count branch.

Thus specifying L and M implicitly fixes a UV cutoff, while specifying L and cutoffs implicitly fixes the count. These descriptions coincide for a symmetric contiguous grid containing zero, but not for arbitrary selected subsets or positive-IR bands. Continuous cutoffs are not uniquely recoverable from a mode set: with IR zero, the same $\mathcal{K}_M$ is obtained whenever

$$J\Delta k \leq k_{\text{UV}} < (J+1)\Delta k \quad (6)$$

in exact arithmetic. The estimator reports effective radial extrema, without claiming that the requested cutoff originally equalled an extremum. It reports the smallest nonzero momentum separately: effective IR zero does not mean zero spacing between the nonzero modes.

Additional mode selection. Selection is applied after the base grid is resolved. With $w_p = \text{photon_window}$ and $w_a = \text{atom_window}$ it retains the union

$$|k - k_0| \leq w_p \sigma_k \quad \text{or} \quad |k - \omega_0| \leq w_a \sigma_k \quad \text{or} \quad |k + \omega_0| \leq w_a \sigma_k. \quad (7)$$

Each comparison has an absolute 10^{-12} momentum tolerance. Each individually empty window adds its nearest available mode; ties select the first mode in the sorted array. A zero-radius window therefore still retains a nearest mode without an exact resonance. Factors must be finite and nonnegative. Selection can yield asymmetry, holes, an even selected count, or no zero mode. These follow from the requested windows, not a hidden modification of the base-grid convention.

The packet is then rebuilt and normalized on the selected grid. A comparison therefore tests both preparation and dynamics. For a selected subset $S \subseteq \mathcal{K}$, with c_m the normalized unselected packet,

$$p_S = \sum_{m \in S} |c_m|^2, \quad c_m^{(S)} = c_m / \sqrt{p_S} \quad (m \in S), \quad F_{\text{state}}(0) = p_S, \quad F_{\text{atom}}(0) = 1 \quad (8)$$

for identical number preparations with $1 \leq n \leq N$, defined below. For a coherent preparation, renormalizing the selected packet also changes its individual mode amplitudes. The fixed-product projection formula below does not automatically apply across such different grids.

Three photon bases and their ordering. A photon configuration is $\mathbf{n} \in \mathbb{N}^M$. With $N = \mathbf{n_max}$,

$$\mathcal{B}_{\text{truncated}} = \{\mathbf{0}\} \cup \{n\mathbf{e}_m : 1 \leq m \leq M, 1 \leq n \leq N\}, \quad (9)$$

$$\mathcal{B}_{\text{total}} = \{\mathbf{n} : \sum_m n_m \leq N\}, \quad (10)$$

$$\mathcal{B}_{\text{full}} = \{\mathbf{n} : 0 \leq n_m \leq N \quad \forall m\}. \quad (11)$$

The string names are `truncated`, `full+totalcap` and `full`. The TLS is a separate $\mathbb{C}^2$ factor in all three. There is no supplemental atomic oscillator or `truncated+atom` basis. At fixed grid and cap,

$$\mathcal{B}_{\text{truncated}} \subseteq \mathcal{B}_{\text{total}} \subseteq \mathcal{B}_{\text{full}}. \quad (12)$$

The joint ket dimensions are

$$d_{\text{truncated}} = 2(1 + MN), \quad d_{\text{total}} = 2 \binom{M+N}{N}, \quad d_{\text{full}} = 2(N+1)^M. \quad (13)$$

Configurations with total r are weak compositions of r into M entries, numbering $\binom{M+r-1}{r}$. Summing over $0 \leq r \leq N$ gives $\binom{M+N}{N}$. At $N = 0$ every basis has one vacuum and two TLS components; at $N = 1$ truncated and total-cap coincide. For $M = 1$ all three coincide.

Indices are representation dependent. For $M = 2, N = 2$, the exact source order is:

i	<code>truncated</code>	<code>full+totalcap</code>	<code>full</code>
0	(0, 0)	(0, 0)	(0, 0)
1	(1, 0)	(1, 0)	(0, 1)
2	(2, 0)	(0, 1)	(0, 2)
3	(0, 1)	(2, 0)	(1, 0)
4	(0, 2)	(1, 1)	(1, 1)
5	—	(0, 2)	(1, 2)
6	—	—	(2, 0)
7	—	—	(2, 1)
8	—	—	(2, 2)

Total-cap enumerates combinations of mode indices with replacement and converts their multiplicities to occupations. Full uses a Cartesian product with the last entry changing fastest. In every representation, $s = 0$ denotes g , $s = 1$ denotes e , and

$$q = 2i + s, \quad v_{2i} = C_{i,g}, \quad v_{2i+1} = C_{i,e}, \quad |\psi\rangle = \sum_{i,s} C_{i,s} |\mathbf{n}_i; s\rangle. \quad (14)$$

An occupation-to-index dictionary supplies targets. Equal raw indices in different bases do not generally mean equal physical configurations.

1.2 Preparation, projected Hamiltonian and evolution

Gaussian coefficients and number preparation. For $k_0, x_0 \in \mathbb{R}$ and $\sigma_k > 0$,

$$\tilde{c}_m = \exp\left[-\frac{(k_m - k_0)^2}{4\sigma_k^2}\right] e^{-ik_m x_0}, \quad c_m = \frac{\tilde{c}_m}{\sqrt{\sum_\ell |\tilde{c}_\ell|^2}}. \quad (15)$$

All selected signed modes enter. There is no right-moving-only option: positive k_0 specifies a centre, not a filter. In the continuous ideal Gaussian, $|\tilde{c}(k)|^2$ has standard deviation σ_k . The actual finite-band moments are

$$\langle k \rangle_c = \sum_m k_m |c_m|^2, \quad (\Delta k_c)^2 = \sum_m (k_m - \langle k \rangle_c)^2 |c_m|^2, \quad (16)$$

and need not equal k_0 and σ_k^2 . The code subtracts the maximum real exponent before exponentiation. This common positive scale cancels at normalization and avoids an entirely underflowed packet for a distant centre.

The TLS preparation is one of

$$|a_0\rangle \in \left\{ |g\rangle, |e\rangle, (|g\rangle + |e\rangle)/\sqrt{2}, (|g\rangle - |e\rangle)/\sqrt{2} \right\}, \quad (17)$$

encoded by `initial_state='g', 'e', '+'` or `'-'` in the atomic dictionary, with default g . For `state='number'`,

$$|\psi_0^{(n)}\rangle = \begin{cases} |0\rangle \otimes |a_0\rangle, & n = 0, \\ \sum_m c_m |n\mathbf{e}_m\rangle \otimes |a_0\rangle, & 1 \leq n \leq N. \end{cases} \quad (18)$$

The default is this number preparation with $n = 1$. Orthogonality of the configurations proves normalization for $n > 0$. At $n = 0$ the vacuum is constructed once; summing it once per mode would duplicate one physical state.

For $n > 1$, all n photons occupy one mode at a time. A Fock state of a collective packet mode $b^\dagger = \sum_m c_m a_m^\dagger$ would instead be

$$\frac{(b^\dagger)^n}{\sqrt{n!}} |0\rangle = \sum_{\sum_m n_m = n} \sqrt{\frac{n!}{\prod_m n_m!}} \prod_m c_m^{n_m} |\mathbf{n}\rangle. \quad (19)$$

For two modes and $n = 2$, this has amplitudes $c_1^2, \sqrt{2}c_1c_2, c_2^2$ on $(2, 0), (1, 1), (0, 2)$; the implementation has c_1, c_2 on $(2, 0), (0, 2)$ only. The historical preparation is preserved, so higher- n results must use its actual definition.

Coherent projection and its retained weight. For `state='coherent'`, α is finite and complex, and

$$|\alpha\rangle = \bigotimes_m |\alpha c_m\rangle, \quad b_n = e^{-|\alpha|^2/2} \prod_m \frac{(\alpha c_m)^{n_m}}{\sqrt{n_m!}}. \quad (20)$$

The total unprojected mean photon number is $\lambda = |\alpha|^2$, with individual means $\lambda_m = \lambda |c_m|^2$ and $\sum_m \lambda_m = \lambda$. Let

$$P_{\mathcal{B}} = \sum_{\mathbf{n} \in \mathcal{B}, s} |\mathbf{n}; s\rangle \langle \mathbf{n}; s|, \quad Z_{\mathcal{B}} = \sum_{\mathbf{n} \in \mathcal{B}} |b_{\mathbf{n}}|^2. \quad (21)$$

The actual prepared ket is

$$|\psi_{0,\mathcal{B}}\rangle = \frac{P_{\mathcal{B}}(|\alpha\rangle \otimes |a_0\rangle)}{\sqrt{Z_{\mathcal{B}}}}. \quad (22)$$

The common exponential prefactor is not evaluated in the source because it cancels on normalization. With $E_N(z) = \sum_{r=0}^N z^r / r!$,

$$Z_{\text{full}} = e^{-\lambda} \prod_m E_N(\lambda_m), \quad (23)$$

$$Z_{\text{total}} = e^{-\lambda} E_N(\lambda), \quad (24)$$

$$Z_{\text{truncated}} = e^{-\lambda} \left[1 + \sum_m (E_N(\lambda_m) - 1) \right]. \quad (25)$$

The total-cap formula follows from the multinomial identity

$$\sum_{\sum_m n_m = r} \prod_m \frac{\lambda_m^{n_m}}{n_m!} = \frac{(\sum_m \lambda_m)^r}{r!} = \frac{\lambda^r}{r!}. \quad (26)$$

Truncated counts vacuum once and nonzero occupations on each mode ray. Consequently $0 < Z_{\text{truncated}} \leq Z_{\text{total}} \leq Z_{\text{full}} \leq 1$. At $\alpha = 0$ all weights equal one. If several λ_m are nonzero, increasing N in truncated alone approaches $e^{-\lambda}[1 + \sum_m (e^{\lambda_m} - 1)]$, generally below one: multimode occupations remain omitted at every cap.

For two projections of the *same* coherent product on the same grid,

$$F_{\text{state}}(0) = \frac{Z_{A \cap B}^2}{Z_A Z_B}, \quad Z_{A \cap B} = \sum_{\mathbf{n} \in \mathcal{B}_A \cap \mathcal{B}_B} |b_{\mathbf{n}}|^2, \quad F_{\text{atom}}(0) = 1. \quad (27)$$

If $\mathcal{B}_A \subseteq \mathcal{B}_B$, this is Z_A/Z_B . For two equally weighted modes, $\lambda = 1$ and $N = 2$, the weights are $2.25/e$, $2.5/e$, and $2.640625/e$ for truncated, total-cap and full. Truncated versus total-cap therefore starts at $F_{\text{state}}(0) = 0.9$ before any interaction. All these preparations are field-TLS products, explaining their unit atomic fidelity and zero initial atomic entropy.

Hamiltonian and matrix entries. For finite real D , $\omega_0 \geq 0$ and finite x_{tls} ,

$$H_0 = \sum_m |k_m| a_m^\dagger a_m + \omega_0 |e\rangle \langle e|, \quad (28)$$

$$u_m = \frac{if_m e^{-ik_m x_{\text{tls}}}}{\sqrt{L}}, \quad f_m = \begin{cases} \sqrt{|k_m|}, & \text{sqrt,} \\ 1, & \text{flat,} \end{cases} \quad (29)$$

$$H_{\mathcal{B}} = P_{\mathcal{B}} \left[H_0 + D \sum_m (u_m a_m + u_m^* a_m^\dagger) (\sigma_+ + \sigma_-) \right] P_{\mathcal{B}} \equiv H_{0,\mathcal{B}} + DV_{\mathcal{B}}, \quad (30)$$

where $\sigma_+ = |e\rangle \langle g|$ and $\sigma_- = |g\rangle \langle e|$. The factor i remains at zero TLS position. Creation receives the conjugate annihilation coefficient. Because field and TLS operators commute, $(u_m a_m \sigma_x)^\dagger = u_m^* a_m^\dagger \sigma_x$; the sum is Hermitian, and orthogonal projection preserves this property. In energy units, D is dimensionless for **sqrt** and has units $\sqrt{\text{energy}}$ for **flat**. Equal numerical couplings under the profiles do not describe the same physical model.

For a source column $q = 2i + s$, the assembled entries are

$$(H_0)_{2i+s, 2i+s} = \sum_m |k_m| n_{i,m} + s\omega_0, \quad (31)$$

$$(V_{\mathcal{B}})_{2j+1-s, 2i+s}^{(-)} = u_m \sqrt{n_{i,m}}, \quad \mathbf{n}_j = \mathbf{n}_i - \mathbf{e}_m, \quad (32)$$

$$(V_{\mathcal{B}})_{2j+1-s, 2i+s}^{(+)} = u_m^* \sqrt{n_{i,m} + 1}, \quad \mathbf{n}_j = \mathbf{n}_i + \mathbf{e}_m \in \mathcal{B}. \quad (33)$$

A descending transition needs $n_{i,m} > 0$. All bases are downward closed, so the target then exists. An ascending target is looked up and omitted if absent, realizing exactly $P_{\mathcal{B}} H P_{\mathcal{B}}$. No omitted target is replaced by another oscillator. A transition and its reverse are both retained or both absent, with conjugate coefficients.

For example, creation in mode two from $(1, 0)$ targets $(1, 1)$: retained in total-cap/full at $N = 2$, absent in truncated. Creation from $(2, 0)$ into $(2, 1)$ is retained only in full. Shared entries have identical coefficients, but different transition graphs can produce different dynamics. Projected ladder operators are not canonical at the boundary. For one mode,

$$a_N |n\rangle = \sqrt{n} |n-1\rangle, \quad a_N^\dagger |N\rangle = 0, \quad [a_N, a_N^\dagger] = I - (N+1) |N\rangle \langle N|. \quad (34)$$

A calculation can be exact for this finite model without proving convergence to unbounded Fock space. Assembly costs $O(Md)$, rather than scanning all pairs.

RWA, parity and the zero mode. Under RWA only annihilation with excitation and creation with de-excitation remain:

$$H_{\mathcal{B}, \text{RWA}} = P_{\mathcal{B}} \left[H_0 + D \sum_m (u_m a_m \sigma_+ + u_m^* a_m^\dagger \sigma_-) \right] P_{\mathcal{B}}. \quad (35)$$

With $\mathcal{N}_f = \sum_m n_m$, $\mathcal{N} = \mathcal{N}_f + |e\rangle \langle e|$ and $\Pi = (-1)^{\mathcal{N}}$,

$$[H_{\mathcal{B}, \text{RWA}}, \mathcal{N}] = 0, \quad [H_{\mathcal{B}}, \Pi] = 0. \quad (36)$$

RWA transitions change photon and atomic excitation by opposite units. Counter-rotating terms change their sum by ± 2 , preserving its parity. Occupation projectors commute with the diagonal symmetry operators, so these identities survive all three projections.

For the retained zero mode and **sqrt**,

$$\omega(0) = f(0) = 0, \quad [H_{\mathcal{B}}, n_0] = 0. \quad (37)$$

It can carry initial norm without free energy or direct TLS coupling. In full, fixing $n_0 = r$ leaves the same nonzero-mode Hamiltonian for each r . In total-cap the corresponding sector instead obeys

$$\sum_{m: k_m \neq 0} n_m \leq N - r. \quad (38)$$

A conserved zero-mode occupation thus changes the available nonzero photon space. In truncated, positive zero-mode occupation cannot coexist with occupation of another mode. Deleting zero changes preparation normalization and these finite spaces even though $f(0) = 0$. For flat, $f(0) = 1$ and the zero mode couples, so n_0 is generally no longer conserved. At $N = 0$ all interactions are projected away. At explicit $M = 1$ with sqrt the only mode is zero and only TLS free evolution remains.

Propagation, output sampling and numerical normalization. The engine solves the time-independent equation

$$i \frac{d}{dt} |\psi(t)\rangle = H_B |\psi(t)\rangle, \quad |\psi(t)\rangle = e^{-iH_B t} |\psi_0\rangle. \quad (39)$$

The default integration method used by this engine is BDF. Relative and absolute tolerances are `rtol=1e-9` and `atol=1e-11`. Output renormalization is disabled (`normalize_output=False`). Output `dt` is not the adaptive solver's internal step. Nominal outputs are $0, dt, \dots, Jdt$, $J = \lfloor T/dt \rfloor$, with T appended if needed. Endpoint roundoff is handled at 10^{-12} ; both zero and exact T are always included. If `dt` exceeds T , the list is $[0, T]$.

Exact evolution conserves $\|v\|^2$ and $\langle H_B \rangle$. The reported norm is the squared norm, not its square root. Coefficients, populations and energies are not renormalized after integration. Entropy and fidelity normalize internally. If $v_{\text{num}} = (1 + \epsilon)e^{i\theta}v_{\text{exact}}$, its fidelity is one after normalization but its raw norm is $|1 + \epsilon|^2$. Fidelity alone therefore does not certify norm accuracy.

With `store_state=True`, solver history and coefficient arrays contain all outputs. With `store_state=False`, only the final ket and final coefficient arrays are retained; the output-time list still exists. Time-series fidelity requires both histories. Diagnostic methods accept `t=None` for all retained states, `t=-1` for final, or a physical time in $[0, T]$. With history, a time selects the nearest output without interpolation; a final-only run rejects earlier times. The wavefunction method instead takes an output *index*.

1.3 Observables, exact fidelities and physical interpretation

TLS reduction and entanglement. Tracing equation (14) over field configurations gives

$$\rho_a = C^\top C^*, \quad (\rho_a)_{ss'} = \sum_i C_{i,s} C_{i,s'}^*, \quad P_e = \sum_i |C_{i,e}|^2, \quad \text{Tr } \rho_a = \|v\|^2. \quad (40)$$

The transpose is intentionally not an adjoint: rows of C are photon configurations and columns are TLS labels. For $\hat{\rho}_a = \rho_a / \|v\|^2 = \begin{pmatrix} p_g & z \\ z^* & p_e \end{pmatrix}$,

$$\lambda_{\pm} = \frac{1 \pm \sqrt{(p_g - p_e)^2 + 4|z|^2}}{2}, \quad S_a = - \sum_{r=\pm} \lambda_r \log \lambda_r. \quad (41)$$

The logarithm is natural and $0 \log 0 = 0$, so $0 \leq S_a \leq \log 2$. For a global pure ket this measures field-TLS entanglement: zero for a product, maximal for a maximally mixed TLS reduction. The excited-probability method returns the raw diagonal of ρ_a , exposing normalization drift.

Photon sectors and Fourier reconstruction. Directional populations refer specifically to one photon and ground-state TLS:

$$P_{+,1g} = \sum_{m:k_m > 0} |C_{i(e_m),g}|^2, \quad P_{-,1g} = \sum_{m:k_m < 0} |C_{i(e_m),g}|^2, \quad (42)$$

$$P_{0,1g} = \sum_{m:k_m = 0} |C_{i(e_m),g}|^2, \quad p_\nu = \sum_{i: \sum_m n_{i,m} = \nu} \sum_s |C_{i,s}|^2. \quad (43)$$

Their keys are `p_transmitted`, `p_reflected`, `p_zero_1g` and `p_<nu>_photon`. Exact sum rules are

$$P_{+,1g} + P_{-,1g} + P_{0,1g} = P_{1g} \leq p_1, \quad \sum_\nu p_\nu = \|v\|^2. \quad (44)$$

The difference $p_1 - P_{1g}$ is the one-photon, excited-TLS population. Photon totals range through MN in full and N in the other two bases. Directional populations vanish at $N = 0$.

The extracted function is

$$\phi_{1g}(x, t) = \frac{1}{\sqrt{L}} \sum_m C_{i(e_m), g}(t) e^{ik_m x}, \quad \int_{x_*}^{x_*+L} |\phi_{1g}(x, t)|^2 dx = P_{1g}(t). \quad (45)$$

This is a component of the ket, not a density of all photons in a multiphoton state. Its integrated weight may be less than one. The integral uses orthogonality from equation (1); a plotting mesh approximates it. The function is periodic. Under free evolution a positive-momentum component obeys $\phi(x, t) = \phi(x - t, 0)$ and a negative-momentum component obeys $\phi(x, t) = \phi(x + t, 0)$. Both directions can be present in the packet.

The imposed spatial phases give, for an input in the $1g$ sector,

$$(H\psi)_{0,e} = \frac{iD}{\sqrt{L}} \sum_m f_m C_{i(e_m), g} e^{-ik_m x_{\text{tIs}}}, \quad (46)$$

$$\text{for flat: } (H\psi)_{0,e} = iD\phi_{1g}(-x_{\text{tIs}}). \quad (47)$$

For sqrt the same coordinate enters the field weighted by $\sqrt{|k_m|}$. The scattering plot therefore marks $-x_{\text{tIs}}$ in its positive-Fourier coordinate. The parameter remains the one in the requested annihilation phase. Identifying it directly with the plotted coordinate would require the opposite annihilation-phase sign; at zero they coincide. A general non-RWA input can have additional contributions to this target from other sectors.

Energy partitions and their sum rules. Start with the total energy. The primary quantity is the expectation of the *same projected Hamiltonian that generates the evolution*. No partition is needed to compute it. To make every index explicit, write

$$\mathcal{H}_{\mathcal{B}} = \text{span}\{|\mathbf{n}_i; s\rangle : i = 0, \dots, B-1, s = 0, 1\}, \quad d = 2B, \quad (48)$$

$$|q\rangle \equiv |\mathbf{n}_i; s\rangle, \quad q = 2i + s, \quad v_q(t) = \langle q|\psi(t)\rangle = C_{i,s}(t), \quad (49)$$

$$H_{rc} \equiv \langle r|H_{\mathcal{B}}|c\rangle, \quad r, c = 0, \dots, d-1. \quad (50)$$

Thus r labels a *target basis row*, c a *source basis column*; neither is a photon-mode index. Below, mode indices follow the Python arrays, $m = 0, \dots, M-1$; this only shifts the earlier $1, \dots, M$ labelling. For an operator A and an ideally normalized state, the notation is

$$\langle A \rangle_t \equiv \langle \psi(t) | A | \psi(t) \rangle. \quad (51)$$

Time arguments are omitted when every quantity is evaluated at the same t . In particular,

$$E(t) \equiv \langle \psi(t) | H_{\mathcal{B}} | \psi(t) \rangle = v(t)^\dagger H_{\mathcal{B}} v(t) \quad (52)$$

$$= \sum_{r=0}^{d-1} \sum_{c=0}^{d-1} v_r^*(t) H_{rc} v_c(t) = \sum_{r=0}^{d-1} v_r^*(t) (H_{\mathcal{B}} v(t))_r. \quad (53)$$

The last equality is just matrix multiplication: $(H_{\mathcal{B}} v)_r = \sum_c H_{rc} v_c$. Hermiticity implies

$$(v^\dagger H_{\mathcal{B}} v)^* = v^\dagger H_{\mathcal{B}}^\dagger v = v^\dagger H_{\mathcal{B}} v, \quad E(t) \in \mathbb{R}. \quad (54)$$

Taking a real part in the code removes numerical imaginary roundoff; it is not an additional approximation to the Hermitian expectation.

Here $\partial_t H_{\mathcal{B}} = 0$ and $\dot{\psi} = -iH_{\mathcal{B}}\psi$, so

$$\frac{dE}{dt} = \langle \dot{\psi} | H_{\mathcal{B}} | \psi \rangle + \langle \psi | H_{\mathcal{B}} | \dot{\psi} \rangle \quad (55)$$

$$= i \langle \psi | H_{\mathcal{B}}^2 | \psi \rangle - i \langle \psi | H_{\mathcal{B}}^2 | \psi \rangle = 0, \quad (56)$$

$$E(t) = E(0). \quad (57)$$

Consequently, plotting $E(t)$ is first a conservation diagnostic. It does not by itself show energy exchange between the free field, the TLS and the interaction, since those contributions may vary while their sum stays fixed. This conservation identity holds for each finite projected model separately; it does not imply that different projections produce identical trajectories.

Free and interaction operators, with the projection specified. Let I_f be the identity on the photon space before projection and $I_a = |g\rangle\langle g| + |e\rangle\langle e|$ the TLS identity. The joint projector is

$$P_{\mathcal{B}} = \sum_{i=0}^{B-1} \sum_{s=0}^1 |\mathbf{n}_i; s\rangle \langle \mathbf{n}_i; s|, \quad P_{\mathcal{B}}^2 = P_{\mathcal{B}} = P_{\mathcal{B}}^\dagger. \quad (58)$$

The following operators are restricted to $\mathcal{H}_{\mathcal{B}}$:

$$N_m \equiv P_{\mathcal{B}}(a_m^\dagger a_m \otimes I_a)P_{\mathcal{B}}, \quad N_m |\mathbf{n}_i; s\rangle = n_{i,m} |\mathbf{n}_i; s\rangle, \quad (59)$$

$$P_a^{(e)} \equiv P_{\mathcal{B}}(I_f \otimes |e\rangle\langle e|)P_{\mathcal{B}}, \quad P_a^{(e)} |\mathbf{n}_i; s\rangle = s |\mathbf{n}_i; s\rangle, \quad (60)$$

$$H_{f,m} \equiv \omega_m N_m, \quad \omega_m = |k_m|, \quad H_f \equiv \sum_{m=0}^{M-1} H_{f,m}, \quad (61)$$

$$H_a \equiv \omega_0 P_a^{(e)}, \quad H_{0,\mathcal{B}} = H_f + H_a. \quad (62)$$

The projector $P_a^{(e)}$ is an operator; $P_e(t)$ is its scalar expectation, the excited population. Explicitly,

$$P_e(t) = \langle P_a^{(e)} \rangle = \sum_i |C_{i,1}|^2, \quad \langle N_m \rangle = \sum_{i,s} n_{i,m} |C_{i,s}|^2. \quad (63)$$

To remove any ambiguity about V_m , define it *before* taking its expectation. The coefficients and TLS operators used here are

$$\sigma_+ = |e\rangle\langle g|, \quad \sigma_- = |g\rangle\langle e|, \quad \sigma_x = \sigma_+ + \sigma_-, \quad u_m = \frac{if_m e^{-ik_m x_{\text{tls}}}}{\sqrt{L}}, \quad f_m = \begin{cases} \sqrt{|k_m|}, & \text{sqrt,} \\ 1, & \text{flat.} \end{cases} \quad (64)$$

Without RWA the mode-labelled interaction operator is

$$V_m \equiv P_{\mathcal{B}}[(u_m a_m + u_m^* a_m^\dagger) \otimes \sigma_x]P_{\mathcal{B}} \quad (65)$$

$$= \frac{if_m}{\sqrt{L}} P_{\mathcal{B}}[(e^{-ik_m x_{\text{tls}}} a_m - e^{ik_m x_{\text{tls}}} a_m^\dagger) \otimes \sigma_x]P_{\mathcal{B}}. \quad (66)$$

Under RWA its definition is instead

$$V_m \equiv P_{\mathcal{B}}(u_m a_m \otimes \sigma_+ + u_m^* a_m^\dagger \otimes \sigma_-)P_{\mathcal{B}}. \quad (67)$$

In either case,

$$V_m^\dagger = V_m, \quad V_{\mathcal{B}} = \sum_{m=0}^{M-1} V_m, \quad H_{\text{int}} \equiv DV_{\mathcal{B}}, \quad H_{\mathcal{B}} = H_f + H_a + H_{\text{int}}. \quad (68)$$

V_m is thus an *operator on the joint field-TLS space*, containing only interactions that change occupation of photon mode m . It already contains the bosonic operators, the TLS transition, f_m , $L^{-1/2}$ and the phase including i ; it does *not* contain D . The energy term is DV_m . V_m is neither an occupation number nor an energy expectation. Projection omits transitions whose target is outside the chosen basis, as in equation (33).

For completeness, let $R = 0$ without RWA and $R = 1$ with RWA, and set

$$\chi_-(s) = 1 - Rs, \quad \chi_+(s) = 1 - R(1 - s). \quad (69)$$

These select source $s = 0$ for annihilation and source $s = 1$ for creation when RWA is active. For $r = 2j + s'$ and $c = 2i + s$,

$$(V_m)_{rc} = \delta_{s',1-s} \begin{cases} \chi_-(s) u_m \sqrt{n_{i,m}}, & \mathbf{n}_j = \mathbf{n}_i - \mathbf{e}_m, n_{i,m} > 0, \\ \chi_+(s) u_m^* \sqrt{n_{i,m} + 1}, & \mathbf{n}_j = \mathbf{n}_i + \mathbf{e}_m \in \mathcal{B}, \\ 0, & \text{otherwise.} \end{cases} \quad (70)$$

Here $\mathbf{e}_m$ has occupation one in mode m and zero in every other mode. This gives an explicit matrix definition of the same operator. The reverse transition has the conjugate coefficient because $\chi_+(1-s) = \chi_-(s)$, and both targets lie in the retained basis.

The mode partition is a specified allocation of interaction energy. Define the real scalar contributions

$$F_m(t) \equiv \langle H_{f,m} \rangle = \omega_m \sum_{i,s} n_{i,m} |C_{i,s}(t)|^2, \quad (71)$$

$$W_m(t) \equiv \langle DV_m \rangle = Dv^\dagger V_m v, \quad (72)$$

$$E_a(t) \equiv \langle H_a \rangle = \omega_0 \sum_i |C_{i,1}(t)|^2, \quad (73)$$

$$E_f(t) \equiv \sum_m F_m(t), \quad E_{\text{int}}(t) \equiv \sum_m W_m(t). \quad (74)$$

The code's “energy per mode” is

$$E_m(t) \equiv F_m(t) + W_m(t) = \langle \psi | (H_{f,m} + DV_m) | \psi \rangle, \quad \sum_m E_m(t) + E_a(t) = E(t). \quad (75)$$

The sum rule follows by substituting the operator identity (68); it is not a separately assumed conservation law. F_m is nonnegative for a normalized state, whereas W_m can have either sign. E_m includes a field-TLS interaction and is therefore not the bare photon energy $\omega_m \langle N_m \rangle$.

The attribution of that interaction to the mode-labelled term is a convention. For example, any fixed real λ_m would also yield the exact decomposition

$$\tilde{H}_m = H_{f,m} + \lambda_m DV_m, \quad \tilde{H}_a = H_a + \sum_m (1 - \lambda_m) DV_m, \quad \sum_m \tilde{H}_m + \tilde{H}_a = H_{\mathcal{B}}. \quad (76)$$

The implemented convention is $\lambda_m = 1$. The separately displayed TLS energy is its *bare* term E_a , while all interaction is in the mode columns. These pieces need not be individually conserved. For a fixed Hermitian component A ,

$$\frac{d}{dt} \langle A \rangle = i \langle [H_{\mathcal{B}}, A] \rangle, \quad \frac{d}{dt} (E_f + E_a + E_{\text{int}}) = 0. \quad (77)$$

The commutators generally do not vanish separately.

What the stored quadruple means, and the derivation of the sparse sum. A sparse matrix can be represented by a list of row/column/value records. Here a fourth field records which mode generated the transition. Let N_{tr} be the length of that list, and define for $\ell = 0, \dots, N_{\text{tr}} - 1$

$$(r_\ell, c_\ell, z_\ell, m_\ell), \quad z_\ell \equiv (V_{m_\ell})_{r_\ell c_\ell}. \quad (78)$$

This is precisely what the earlier shorthand (r, c, V_{rc}, m) meant: r and c are joint ket indices, V_{rc} is a complex matrix coefficient without D , and m is a mode label. Each off-diagonal interaction connects configurations differing in only one mode, so that mode is unique and the recorded z_ℓ is also the corresponding entry of $V_{\mathcal{B}}$. The record is not an additional physical variable or a state.

Python array entry	Mathematical symbol	Type and range
<code>transition_rows[1]</code>	r_ℓ	Integer in $0, \dots, d-1$
<code>transition_cols[1]</code>	c_ℓ	Integer in $0, \dots, d-1$
<code>transition_values[1]</code>	z_ℓ	Complex coefficient of $V_{\mathcal{B}}$
<code>transition_modes[1]</code>	m_ℓ	Integer in $0, \dots, M-1$

For example, for the source configuration $\mathbf{n}_i = \mathbf{e}_m$ and TLS g , annihilation to vacuum with TLS e records

$$c_\ell = 2i, \quad r_\ell = 2i(\mathbf{0}) + 1, \quad z_\ell = u_m, \quad m_\ell = m. \quad (79)$$

Here $i(\mathbf{0})$ is the photon-vacuum index; the packet amplitude does not enter this stored matrix value. It enters the expectation through v_c .

Define δ_{m,m_ℓ} as one when the tags match and zero otherwise. The exact matrix reconstruction and its expectation are

$$V_m = \sum_{\ell=0}^{N_{\text{tr}}-1} \delta_{m,m_\ell} z_\ell |r_\ell\rangle \langle c_\ell|, \quad (80)$$

$$v^\dagger V_m v = \sum_{\ell=0}^{N_{\text{tr}}-1} \delta_{m,m_\ell} z_\ell (v^\dagger |r_\ell\rangle) (\langle c_\ell| v) \quad (81)$$

$$= \sum_{\ell=0}^{N_{\text{tr}}-1} \delta_{m,m_\ell} v_{r_\ell}^* z_\ell v_{c_\ell}. \quad (82)$$

Since D is real and V_m is Hermitian, its real energy contribution is

$$W_m(t) = \sum_{\ell=0}^{N_{\text{tr}}-1} \delta_{m,m_\ell} \Re \left[v_{r_\ell}^*(t) D z_\ell v_{c_\ell}(t) \right]. \quad (83)$$

This is the matrix expectation expanded in recorded entries. It follows from $v^\dagger V_m v$ and the resolution into $|r\rangle \langle c|$; no additional energy formula has been postulated. For a coefficient array $\mathbf{v}$, the equivalent NumPy operations are:

```

1 h = experiment.hamiltonian
2 D = experiment.param_atom['D']
3 M = experiment.n_modes
4 contributions = np.real(
5     v[h.transition_rows].conj() * D * h.transition_values
6     * v[h.transition_cols])
7 interaction = np.bincount(
8     h.transition_modes, weights=contributions, minlength=M)

```

The second operation is the weighted `np.bincount` in the actual code.

Both directed records of a Hermitian pair are present:

$$(r, c, z, m) \quad \text{and} \quad (c, r, z^*, m), \quad (84)$$

$$v_r^* D z v_c + v_c^* D z^* v_r = 2\Re(v_r^* D z v_c). \quad (85)$$

Thus one can either sum both recorded directions as in (83), or sum one orientation and multiply its real part by two. Multiplying the already two-direction sum by two would be incorrect. Zero-valued records contribute zero.

An equivalent expression makes the amplitudes and the phase explicit. Let $j(i, m)$ index $\mathbf{n}_i - \mathbf{e}_m$.

Summing only descending records gives

$$W_m = 2D\Re \sum_{\substack{i,s \\ n_{i,m}>0}} \chi_-(s) u_m \sqrt{n_{i,m}} C_{j(i,m),1-s}^* C_{i,s} \quad (86)$$

$$= -\frac{2Df_m}{\sqrt{L}} \Im \sum_{\substack{i,s \\ n_{i,m}>0}} \chi_-(s) e^{-ik_m x_{\text{tls}}} \sqrt{n_{i,m}} C_{j(i,m),1-s}^* C_{i,s}. \quad (87)$$

The second line uses $\Re(iz) = -\Im z$. Interaction energy depends on relative phases and field-TLS coherences, not just occupation probabilities. For the fixed-number preparation (18), all amplitudes in the neighbouring photon totals are initially zero, so

$$W_m(0) = 0, \quad E(0) = n \sum_m \omega_m |c_m|^2 + \omega_0 |\langle e|a_0\rangle|^2 \quad (88)$$

for $n \geq 1$. At $n = 0$ the field term is zero. Coherent preparations can instead contain the coherences in (87) initially.

Photon-number partitions: projectors, block matrices and derivation. This is a second way of partitioning the *same* total energy. It does not refer to mode m . Define the photon total of a joint basis index and the associated projector by

$$\nu(q) \equiv \sum_{m=0}^{M-1} n_{i,m}, \quad i = \lfloor q/2 \rfloor, \quad (89)$$

$$Q_\nu \equiv \sum_{q:\nu(q)=\nu} |q\rangle \langle q|, \quad Q_\nu |q\rangle = \mathbf{1}_{\nu(q)=\nu} |q\rangle, \quad (90)$$

$$Q_\nu^\dagger = Q_\nu, \quad Q_\nu Q_\mu = \delta_{\nu\mu} Q_\nu, \quad \sum_{\nu=0}^{\nu_{\max}} Q_\nu = I_{\mathcal{B}}. \quad (91)$$

Here $I_{\mathcal{B}}$ is the identity on the retained joint space, $\nu_{\max} = \max_i \sum_m n_{i,m}$, equal to N in total-cap/truncated and MN in full. Q_ν selects photon number alone; it keeps both TLS states, so ν is not the RWA excitation number $\mathcal{N}$. Define the sector ket, its weight and the corresponding Hamiltonian blocks:

$$|\psi_\nu\rangle = Q_\nu |\psi\rangle, \quad p_\nu = \langle \psi_\nu | \psi_\nu \rangle, \quad H_{\nu\mu} = Q_\nu H_{\mathcal{B}} Q_\mu. \quad (92)$$

In general ψ_ν is not normalized, and $H_{\nu\mu}$ maps sector μ to sector ν . Inserting the projector resolution on both sides of $H_{\mathcal{B}}$ gives

$$E = \sum_{\nu,\mu} \langle \psi_\nu | H_{\mathcal{B}} | \psi_\mu \rangle. \quad (93)$$

The terms with $\nu \neq \mu$ must not be discarded: they are coherences between photon sectors, and the interaction changes photon total by one. Specifically,

$$Q_\nu H_{0,\mathcal{B}} Q_\mu = 0 \quad (\nu \neq \mu), \quad Q_\nu V_{\mathcal{B}} Q_\nu = 0, \quad Q_\nu V_{\mathcal{B}} Q_\mu = 0 \quad (|\nu - \mu| \neq 1). \quad (94)$$

These identities hold with or without RWA; the TLS selection rules change, but each retained interaction still creates or annihilates one photon.

The implemented row-based energy assigns to sector ν

$$E_\nu \equiv \Re \langle \psi | Q_\nu H_{\mathcal{B}} | \psi \rangle = \sum_{q:\nu(q)=\nu} \Re [v_q^* (H_{\mathcal{B}} v)_q]. \quad (95)$$

Why the real part, and why this operator ordering? The projector acts on the *row* of the matrix product, giving the rightmost sum. Since Q_ν and $H_{\mathcal{B}}$ need not commute, $Q_\nu H_{\mathcal{B}}$ is not generally Hermitian. Let $z_\nu = \langle \psi | Q_\nu H_{\mathcal{B}} | \psi \rangle$; then

$$z_\nu^* = \langle \psi | (Q_\nu H_{\mathcal{B}})^\dagger | \psi \rangle = \langle \psi | H_{\mathcal{B}} Q_\nu | \psi \rangle, \quad \Re z_\nu = \frac{1}{2}(z_\nu + z_\nu^*). \quad (96)$$

Consequently this row formula is exactly the expectation of the Hermitian operator A_ν defined below:

$$A_\nu \equiv \frac{1}{2}(Q_\nu H_{\mathcal{B}} + H_{\mathcal{B}} Q_\nu), \quad A_\nu^\dagger = A_\nu, \quad (97)$$

$$E_\nu = \langle \psi | A_\nu | \psi \rangle, \quad \sum_\nu A_\nu = \frac{1}{2}(I_{\mathcal{B}} H_{\mathcal{B}} + H_{\mathcal{B}} I_{\mathcal{B}}) = H_{\mathcal{B}}, \quad (98)$$

$$\sum_\nu E_\nu = E. \quad (99)$$

This proves the sum rule at the operator level, before considering a particular ket or numerical trajectory. It is a symmetric allocation of cross-sector matrix elements, not an assumption that energy is diagonal in photon number.

To display what each sector actually contains, define

$$F_\nu \equiv \langle \psi_\nu | H_{0,\mathcal{B}} | \psi_\nu \rangle = \sum_{q:\nu(q)=\nu} \epsilon_q |v_q|^2, \quad (100)$$

$$\epsilon_q \equiv \sum_m \omega_m n_{i,m} + s\omega_0, \quad q = 2i + s, \quad (101)$$

$$K_{\nu\mu} \equiv \Re \langle \psi_\nu | DV_{\mathcal{B}} | \psi_\mu \rangle, \quad K_{\nu\mu} = K_{\mu\nu}. \quad (102)$$

Here F_ν contains both bare field and bare TLS energy in that sector. The selection rules above now give, with absent sectors contributing zero,

$$E_\nu = F_\nu + K_{\nu,\nu-1} + K_{\nu,\nu+1}, \quad (103)$$

$$E = \sum_\nu F_\nu + 2 \sum_{\nu=0}^{\nu_{\max}-1} K_{\nu,\nu+1}. \quad (104)$$

Each cross-sector coherence $K_{\nu,\nu+1}$ thus contributes equally to E_ν and $E_{\nu+1}$. The total expectation includes twice that value, just as the two conjugate records in (85). Equivalently, in the recorded-entry notation,

$$E_\nu = \sum_{q:\nu(q)=\nu} \epsilon_q |v_q|^2 + \sum_\ell \mathbf{1}_{\nu(r_\ell)=\nu} \Re[v_{r_\ell}^* D z_\ell v_{c_\ell}]. \quad (105)$$

The sector is determined by the *target row* r_ℓ , whereas the mode partition uses tag m_ℓ . They group the same contributions by different labels.

The result is not a conditional energy after a photon-number measurement. For $p_\nu > 0$, the normalized projected state would be $|\hat{\psi}_\nu\rangle = |\psi_\nu\rangle / \sqrt{p_\nu}$, with

$$E_{\text{cond},\nu} \equiv \langle \hat{\psi}_\nu | H_{\mathcal{B}} | \hat{\psi}_\nu \rangle = \frac{\langle \psi_\nu | H_{\mathcal{B}} | \psi_\nu \rangle}{p_\nu} = \frac{F_\nu}{p_\nu}. \quad (106)$$

The final equality uses $Q_\nu V_{\mathcal{B}} Q_\nu = 0$. Therefore

$$\sum_{\nu:p_\nu>0} p_\nu E_{\text{cond},\nu} = \sum_\nu F_\nu = \langle H_{0,\mathcal{B}} \rangle, \quad \text{generally not } E. \quad (107)$$

Removing coherences by this hypothetical projection removes their interaction energy. The implemented E_ν keeps them via the symmetric allocation. It is neither p_ν , nor νp_ν , nor $p_\nu E_{\text{cond},\nu}$.

An explicit two-amplitude example, including the signs. Choose one retained nonzero mode m and $x_{\text{tls}} = 0$. Evaluate the normalized ket with just two nonzero components

$$|\psi_\varphi\rangle = \frac{|1_m; g\rangle + e^{i\varphi} |0; e\rangle}{\sqrt{2}}, \quad G_m \equiv \frac{Df_m}{\sqrt{L}} \in \mathbb{R}. \quad (108)$$

Other modes may be present but have zero amplitudes in this example. The matrix restricted to these two components, ordered as $(|1_m; g\rangle, |0; e\rangle)$, is

$$H|_{\text{two components}} = \begin{pmatrix} \omega_m & -iG_m \\ iG_m & \omega_0 \end{pmatrix}. \quad (109)$$

This is a submatrix used to evaluate the expectation, not a claim that this pair is a dynamically closed subspace in an arbitrary multimode run. The directed annihilation record connects $|c\rangle = |1_m; g\rangle$ to $|r\rangle = |0; e\rangle$, with $z = u_m = if_m/\sqrt{L}$ and amplitudes $v_r = e^{i\varphi}/\sqrt{2}$, $v_c = 1/\sqrt{2}$. Thus

$$\Re(v_r^* D z v_c) = \Re\left(\frac{iG_m e^{-i\varphi}}{2}\right) = \frac{G_m \sin \varphi}{2}, \quad (110)$$

$$W_m = G_m \sin \varphi, \quad E = \frac{\omega_m + \omega_0}{2} + G_m \sin \varphi. \quad (111)$$

The reverse creation record supplies the other half of W_m . Every photon and TLS population is independent of φ , but the energy is not. The two partitions return

$$E_m = \frac{\omega_m}{2} + G_m \sin \varphi, \quad E_a = \frac{\omega_0}{2}, \quad (112)$$

$$E_{\nu=1} = \frac{\omega_m}{2} + \frac{G_m \sin \varphi}{2}, \quad E_{\nu=0} = \frac{\omega_0}{2} + \frac{G_m \sin \varphi}{2}. \quad (113)$$

Other mode/sector contributions vanish. Both sums give (111), although the interaction is allocated differently. For $\omega_m = 1$, $\omega_0 = 3$, $G_m = 1.2$ and $\varphi = -\pi/2$,

$$W_m = -1.2, \quad E_m = -0.7, \quad E_a = 1.5, \quad E_{\nu=1} = -0.1, \quad E_{\nu=0} = 0.9, \quad E = 0.8. \quad (114)$$

A negative component is compatible with a positive total. It represents an interaction contribution, not a negative probability. Setting $\varphi = 0$ keeps the same populations but gives $W_m = 0$ and $E = 2$.

What to calculate for total-energy monitoring, and the code mapping. All algebraic sum rules also hold for an unnormalized numerical vector. If $\eta(t) = v(t)^\dagger v(t)$, distinguish

$$E_{\text{raw}}(t) = \Re[v^\dagger H_B v], \quad E_{\text{normalized}}(t) = \frac{E_{\text{raw}}(t)}{\eta(t)} \quad (\eta > 0). \quad (115)$$

They coincide for exact normalized evolution. `compute_energy` and both partitions return raw quadratic expressions. To obtain normalized partitions one would divide *all* components by the same η . Dividing only the total would invalidate a comparison of the sum rule. For an already propagated experiment, monitoring the requested quantity needs only:

```

1 observables = experiment.compute_observables()
2 time = observables['time']
3 norm_squared = observables['norm']
4 E_raw = experiment.compute_energy()
5 E_normalized = E_raw / norm_squared
6
7 import matplotlib.pyplot as plt
8 fig, ax = plt.subplots()
9 ax.plot(time, E_raw, label='raw Hamiltonian expectation')
10 ax.axhline(E_raw[0], color='black', linestyle=':', label='initial energy')
11 ax.set(xlabel='t', ylabel='energy')
12 ax.legend()

```

There is no need to compute a mode profile or photon-number profile for this plot. Useful absolute diagnostics are

$$\delta_E = \max_j |E_{\text{raw}}(t_j) - E_{\text{raw}}(0)|, \quad \delta_\eta = \max_j |\eta(t_j) - 1|. \quad (116)$$

An absolute energy error remains defined when $E(0) = 0$, unlike an error normalized by $|E(0)|$. Raw norm, raw energy and tighter solver tolerances should be considered together; a conserved expectation is not a proof of small full-state error.

The exact connections to the executed operations are:

Operation	Quantity defined above
<code>compute_energy</code>	Sparse multiplication followed by $\Re \sum_q v_q^* (Hv)_q$, equation (53).
Mode-profile <code>probability</code>	Array $ C_{i,s} ^2$ with shape $(B, 2)$.
Mode-profile <code>field</code>	$F_m = \omega_m \sum_{i,s} n_{i,m} C_{i,s} ^2$, formed from the occupation matrix and probabilities.
<code>contributions[1]</code>	Directed term $\Re[v_{r_\ell}^* D z_\ell v_{c_\ell}]$, equation (83).
Mode-profile <code>interaction</code>	Weighted count by tag m_ℓ , giving W_m ; the return is $F_m + W_m$ and separate E_a .
Photon-profile <code>energy[q]</code>	Row contribution $\Re[v_q^* (Hv)_q]$. Group pairs of TLS rows and then photon totals to produce E_ν , equation (95).

This maps the displayed total and both heatmaps to their defining operators. The method name `energy_excitations_vec` uses the photon-number axis ν , not a new atomic oscillator or the conserved RWA total excitation.

Implicit embedding in a common full space. Comparisons require a common physical box and consistent conventions. Let $\mathcal{K}_* = \mathcal{K}_A \cup \mathcal{K}_B$ and $N_* \geq \max(N_A, N_B)$; the ambient space is

$$\mathcal{H}_* = \text{span}\{|\mathbf{n}; s\rangle : 0 \leq n_m \leq N_*, k_m \in \mathcal{K}_*, s = g, e\}. \quad (117)$$

A physical mode map μ_X defines

$$(\iota_X \mathbf{n})_{\mu_X(m)} = n_m, \quad (\iota_X \mathbf{n})_\ell = 0 \text{ for absent modes}, \quad \iota_X |\mathbf{n}; s\rangle = |\iota_X \mathbf{n}; s\rangle. \quad (118)$$

Distinct tuples stay orthogonal, so

$$\iota_X^\dagger \iota_X = I_X, \quad \|\iota_X \psi_X\| = \|\psi_X\|. \quad (119)$$

For example, adding $k = 0$ maps $|n_{-1}, n_{+1}; s\rangle$ to $|n_{-1}, 0, n_{+1}; s\rangle$. Positive occupation on that added mode is orthogonal to every such embedded ket. Reordering existing modes only permutes coordinates and cannot change an overlap.

For individually normalized kets,

$$A_{AB} = \langle \iota_A \psi_A | \iota_B \psi_B \rangle = \sum_{\mathbf{n} \in \iota_A \mathcal{B}_A \cap \iota_B \mathcal{B}_B} \sum_s C_{A,\mathbf{n},s}^* C_{B,\mathbf{n},s}, \quad (120)$$

$$F_{\text{state}} = |A_{AB}|^2. \quad (121)$$

The intersection enters because missing amplitudes are zero. No projection and renormalization on the intersection is performed. For orthogonal configurations x, y , take $|\psi_A\rangle = |x\rangle$ and $|\psi_B\rangle = \sqrt{p}|x\rangle + \sqrt{1-p}|y\rangle$. Then

$$F_{\text{state}} = p; \quad \text{renormalizing the common component would incorrectly give 1.} \quad (122)$$

For distinct one-photon modes a, b , the states $(|0; g\rangle + |1_a; g\rangle)/\sqrt{2}$ and $(|0; g\rangle + |1_b; g\rangle)/\sqrt{2}$ share only vacuum. Their overlap is 1/2 and their state fidelity is 1/4.

Keys contain sorted pairs (physical mode index, n_m) for $n_m > 0$; vacuum has the empty key. Each stores two TLS amplitudes. All shared configurations contribute, including several occupied modes. No vector of dimension $2(N_* + 1)^{|\mathcal{K}_*|}$ is created. The union retains the first grid's order and appends absent modes; sorting keys makes that internal order irrelevant. Momentum matching uses absolute tolerance 10^{-12} , zero relative tolerance, and rejects ambiguous identifications.

The time-series interface checks equal L to absolute tolerance 10^{-12} and exactly equal output arrays before pairing states. Low-level `compare_states` has no L or Hamiltonian input, so its caller must ensure physical comparability. Matching numerical momenta from different boxes alone does not certify identical normalized modes.

Atomic fidelity, normalization and bounds. The atomic metric is squared Uhlmann fidelity,

$$F_{\text{atom}} = \left[\text{Tr} \sqrt{\sqrt{\hat{\rho}_{a,A}} \hat{\rho}_{a,B} \sqrt{\hat{\rho}_{a,A}}} \right]^2. \quad (123)$$

The code squares the value returned by `qutip.fidelity`. For qubits,

$$F_{\text{atom}} = \text{Tr}(\hat{\rho}_{a,A} \hat{\rho}_{a,B}) + 2\sqrt{\det \hat{\rho}_{a,A} \det \hat{\rho}_{a,B}}. \quad (124)$$

To derive this, let $X = \sqrt{\rho} \sigma \sqrt{\rho}$ have eigenvalues x_1, x_2 . Then $(\text{Tr} \sqrt{X})^2 = x_1 + x_2 + 2\sqrt{x_1 x_2}$, $\text{Tr} X = \text{Tr}(\rho \sigma)$ and $\det X = \det \rho \det \sigma$. Rank-deficient cases follow by continuity. For Bloch vectors $\mathbf{r}, \mathbf{s}$,

$$F_{\text{atom}} = \frac{1 + \mathbf{r} \cdot \mathbf{s} + \sqrt{(1 - |\mathbf{r}|^2)(1 - |\mathbf{s}|^2)}}{2}. \quad (125)$$

It reduces to squared overlap for pure atomic states, and against a pure $|e\rangle$ to the normalized excited population. Adding vacuum modes leaves the atomic reduction invariant; no ambient allocation is required.

Both metrics normalize each entire finite nonzero ket, not a common subset. Values are clipped to $[0, 1]$ for roundoff overshoots. Raw norm diagnostics remain visible. Monotonicity under the common partial trace yields

$$0 \leq F_{\text{state}} \leq F_{\text{atom}} \leq 1. \quad (126)$$

For instance, $|1_+; g\rangle$ and $|1_-; g\rangle$ have state fidelity zero and atomic fidelity one. No ratio is calculated: a quotient of these metrics is not generally a fidelity of the field reductions.

For normalized embedded pure states, $\rho_A - \rho_B$ has nonzero eigenvalues $\pm \sqrt{1 - |\langle \psi_A | \psi_B \rangle|^2}$ in their two-dimensional span. Therefore, for any common projector, or more generally $0 \leq Q \leq I$,

$$|\text{Tr}[Q(\rho_A - \rho_B)]| \leq \frac{1}{2} \|\rho_A - \rho_B\|_1 = \sqrt{1 - F_{\text{state}}}. \quad (127)$$

This bounds disagreement between normalized populations of two finite calculations, not accuracy against a continuum experiment. For TLS-only measurements an analogous upper bound uses $\sqrt{1 - F_{\text{atom}}}$.

Sample means and scattering interpretation. The coupling sweep and mode selection report

$$\bar{F}_{\text{samples}} = \frac{1}{N_t} \sum_{j=0}^{N_t-1} F(t_j). \quad (128)$$

This differs from final fidelity and a time integral. A trapezoidal time average would instead be

$$\bar{F}_{\text{time}} \simeq \frac{1}{T} \sum_{j=0}^{N_t-2} \frac{F(t_j) + F(t_{j+1})}{2} (t_{j+1} - t_j). \quad (129)$$

Only equation (128) is implemented. Endpoints have different weights even on a uniform grid; a shorter final interval still has an equally weighted output in the sample mean. Initial metrics are stored separately. Cap convergence compares final kets for adjacent entries in the supplied increasing cap list, which need not differ by one.

Scattering supplies finite-packet sector populations and Fourier profiles. If $\sum_{k_m < 0} |c_m|^2 > 0$, final negative-momentum population is not by itself a reflection probability for an exclusively positive-momentum input. Subtracting the initial population need not remove interference. For RWA evolution from one photon and ground TLS, excitation conservation gives

$$P_{+,1g} + P_{-,1g} + P_{0,1g} + P_e = 1 \quad (130)$$

for a normalized state. Without RWA additional sectors can carry norm, so this four-term identity is not general.

Extracting $t(k)$ or $r(k)$ needs identifiable outgoing packets, control of periodic returns and a phase convention relative to free propagation. Counter-rotating terms can require dressed asymptotic states and additional photon channels. The calculated quantities remain finite-time projections; neither fidelity is an S -matrix element.

2 Notebook control, resources and source-line commentary

2.1 Shared configuration and callable Python experiments

Configuration and execution. `ExperimentConfig` collects three physical dictionaries, cutoffs, cap, basis name, RWA, storage choice, grid control and selection windows. Validation occurs when resolving the grid, building the basis/Hamiltonian and preparing the ket. `with_D` returns a copy with a new atomic dictionary. An `Experiment` copies its physical dictionaries before construction, so sweeps do not overwrite the supplied physical configuration. A notebook parameter cell can use:

```

1 import numpy as np
2 from dataclasses import replace
3 from src.xp_config import ExperimentConfig
4 from src.experiment import estimate_config
5 from src.fidelities import fidelities_over_time
6 from experiment.atom_evolution import (
7     run_atom_evolution, plot_atom_evolution,
8 )
9
10 config = ExperimentConfig(
11     param_photon={'k_0': 1., 'sigma_k': .4, 'x_0': -2.,
12                  'state': 'number', 'n': 1},
13     param_atom={'omega_0': 1., 'D': .2, 'L': 2*np.pi,
14                'x_tls': 0., 'coupling': 'sqrt',
15                'initial_state': 'g'},
16     param_time_evol={'T': 4., 'dt': .05,
17                     'rtol': 1e-9, 'atol': 1e-11},
18     n_max=2, CTRL_M_EXPLICIT=True, M=5,
19     truncation='full+totalcap', store_state=True)
20
21 # Estimate every basis that will actually be propagated.
22 for scheme in ('full+totalcap', 'truncated'):
23     estimate_config(replace(config, truncation=scheme))
24 runs = run_atom_evolution(config)
25 metrics = fidelities_over_time(
26     runs['truncated'], runs['full+totalcap'])
27 fig = plot_atom_evolution(runs)

```

For cutoff control, use `CTRL_M_EXPLICIT=False` and `cutoffs={'ir_cutoff': 0., 'uv_cutoff': 2.}` instead. At this L the base grid is identical. Direct execution constructs `Experiment`, calls `propagate_state`, then `compute_observables`; the common experiment helper performs those three operations.

Six experiments and their output contracts. All five migrated Uri notebooks explicitly set `coupling='sqrt'`. The scattering notebook remains alongside them. Modules do not simulate or change the Matplotlib backend on import. Numerical execution and plotting are separate functions, with plotting returning figure objects.

Module	Calculation and main return value
<code>atom_evolution.py</code>	Dictionary of propagated Experiment objects by requested basis. Forces history storage; plots P_e and entropy. The notebook additionally calculates the two fidelities.
<code>energy_profile.py</code>	One experiment with an added energy_profiles dictionary containing both partitions and total energy. Requires history; plots and animation reuse the arrays.
<code>full_cumulative_fidelity.py</code>	Dictionary of increasing caps, retained final-only runs, and initial/final metrics for adjacent cap-list entries. Always uses total-cap.
<code>sweep_D_fidelity.py</code>	Dictionary of physical couplings, candidate names, comparator name, sample means and initial metrics. Forces history.
<code>compare_mode_selection.py</code>	Coupling sweeps with/without selection, and window heatmaps at fixed coupling. Requires history. Comparator is always unselected total-cap.
<code>scattering.py</code>	One experiment with the user's dictionary API; optional observable CSV. Plots $1g$ profiles and directional/zero/TLS/norm populations.

With S candidate schemes, Q coupling values, C caps, and P, A window factor counts, the layouts are:

Result	Array shapes and axes
Atom evolution	Ground/excited coefficient arrays (N_t, B) ; population, entropy and time-series fidelity arrays $(N_t,)$.
Energy	E_modes : (N_t, M) ; E_atom , E_total : $(N_t,)$; E_excitation : $(N_t, \nu_{\max} + 1)$. Its axis counts photons, not photons plus TLS excitation.
Cap convergence	Each metric and its _initial counterpart: $(C - 1,)$. Entry j compares supplied caps j and $j + 1$.
Coupling sweep	Each metric and its _initial counterpart: (S, Q) , scheme before coupling. Unsuffixed metrics are sample means.
Selection	Sweep metrics $(S, 2, Q)$, with middle axis <code>[False, True]</code> . Each _heatmap : (S, P, A) , photon window before atomic window.
Scattering	Observable arrays of length N_t with history, one otherwise; wavefunction array indexed by the supplied spatial samples.

For final-only runs the coefficient arrays have shape $(B,)$, not $(1, B)$; observable methods promote them internally as needed. Diagnostics with `t=None` return a length-one array/list then, whereas an explicit final time returns one scalar, matrix or profile. The mode-profile tuple remains $(k, E_m, 0.0, E_a)$; zero is a compatibility placeholder, not another mode.

Cap convergence compares equal physical/grid inputs with different caps. Its default CLI preparation is coherent, so initial projection differences are expected. The coupling sweep defaults to a propagated total-cap comparator; **reference** can select another finite basis. This is distinct from the ambient full space in equation (117). A candidate with the comparator's scheme reuses its trajectory.

The selection study rebuilds an unselected comparator at each D . Heatmaps use **D_heatmap** when given, otherwise the input configuration's coupling. Their returns contain only the sample means,

not separate initial heatmap metrics; the sweep arrays do contain the `_initial` values. This distinction matters when separating selected-preparation and dynamical effects.

Historical scripts, exports and notebook state. The five `experiments Uri/` paths remain executable delegates. The historical coupling-sweep filename keeps its `g` spelling for path compatibility; arguments and implementation use `D`. Old `key=value` syntax is replaced by `-option value`. Compatible script paths do not imply that historical numerical values require no conversion. From the repository root:

```
1 python -m experiment.scattering --D 0.2 --ctrl-m-explicit --M 5
2 python -m experiment.sweep_D_fidelity --D-values 0.02,0.1,0.2 \
3     --out results/sweep.png
4 python experiments Uri/energy_profile.py --D 0.2 \
5     --out results/energy.png --gif results/energy.gif
```

The CLI defaults to cutoff control; `-M` is active only with `-ctrl-m-explicit`. Both profiles are available, with default `sqrt`; `-RWA` enables RWA. Solver options can be set in the Python time dictionary. Notebooks resolve the root whether the kernel starts there or in `notebooks/`, and keep saved outputs cleared.

With `-out`, a CLI saves a figure and same-stem NPZ with the supplied arrays. Its metadata field is `configuration_json`. JSON stores the base configuration; additional sweep choices and derived grids must be read from accompanying arrays or reconstructed. Complex values are represented as strings. Ordinary exported arrays require no pickle:

```
1 import json
2 with np.load('results/sweep.npz', allow_pickle=False) as data:
3     metadata = json.loads(str(data['configuration_json']))
4     D_values = data['D']
5     F_state_mean = data['F_state']
```

The atom-evolution CLI exports time and excited populations; entropy and notebook fidelity curves are not automatically included in that export. Cap exports omit Python run objects. Optional scattering CSV writes `results/scattering.csv`, overwriting a prior requested export, with one column per observable and no configuration metadata.

2.2 Correspondence between controls and resource costs

What the grid report establishes. The same `resolve_grid` serves propagation and estimation before Fock allocation. Base and selected descriptions contain counts, signed extrema, radial extrema, zero presence, smallest nonzero momentum and integer indices. For integer index set I , the report tests

$$I = \{j : -j_{\max} \leq j \leq j_{\max}, |j| \geq j_{\min}\}, \quad j_{\min} = \min_{j \in I} |j|, \quad j_{\max} = \max_{j \in I} |j|. \quad (131)$$

This is `radial_cutoffs_exact`. If also $j_{\min} = 0$, the count is `equivalent_M`; otherwise that field is `None`. Extrema alone cannot represent asymmetry or holes. At $L = 2\pi$:

Inputs	Grid	Radial exact	Equivalent M
$M = 5$	$\{-2, -1, 0, 1, 2\}$	Yes	5
IR/UV = (0, 2)	$\{-2, -1, 0, 1, 2\}$	Yes	5
IR/UV = (0.2, 2.8)	$\{-2, -1, 1, 2\}$	Yes	None
IR/UV = (0, 0)	$\{0\}$	Yes	1
Selected $\{-2, 0, 2\}$	$\{-2, 0, 2\}$	No	None
Selected $\{0, 1, 2\}$	$\{0, 1, 2\}$	No	None

The first two grids give identical finite dynamics at equal other inputs. A positive-IR band has no allowed equivalent explicit count. The report states this rather than proposing a count that silently inserts zero.

Vector storage and exact dimension growth. A complex128 ket occupies $16d$ bytes. With $S = \text{store_state}$,

$$G_{\text{ket}} = \frac{16d}{2^{30}}, \quad G_{\text{history}} = G_{\text{ket}} \begin{cases} N_t, & S, \\ 1, & \neg S, \end{cases} \quad (132)$$

$$G_{\text{vectors}} \simeq 2G_{\text{history}}, \quad \text{nnz}(H) \leq d(1 + 2M). \quad (133)$$

GiB means 2^{30} bytes. The factor two estimates solver kets and the contiguous coefficient array. Alternating ground/excited views share that array and do not add two more full copies. The initial ket, QuTiP objects, transient conversions and solver workspace can increase storage further.

For $M = 5, N = 2$, joint dimensions are 22, 42, 486 for truncated, total-cap and full. At $T = 4, \text{dt} = 0.05, N_t = 81$ and estimated retained vector bytes are 57 024, 108 864, and 1 259 712. At fixed N and large M ,

$$d_{\text{total}} \sim 2M^N/N!, \quad d_{\text{truncated}} \sim 2NM, \quad d_{\text{full}} = 2(N+1)^M. \quad (134)$$

Polynomial growth at fixed cap does not make high caps cheap; the exact binomial dimension should be evaluated before allocation.

Sparse transitions and excluded costs. Let $A_{\mathcal{B}}$ count photon edges $\mathbf{n} \leftrightarrow \mathbf{n} + \mathbf{e}_m$ in a basis, without TLS labels. For $N \geq 1$,

$$A_{\text{truncated}} = MN, \quad (135)$$

$$A_{\text{total}} = M \binom{M+N-1}{N-1}, \quad (136)$$

$$A_{\text{full}} = MN(N+1)^{M-1}. \quad (137)$$

For total-cap, remove one photon in the changed mode and count configurations with total at most $N-1$. For full, choose the mode, one of its N adjacent levels, and the other $M-1$ occupations. At $N=0$ all counts are zero. The engine records $4A_{\mathcal{B}}$ directed TLS-flipping triplets without RWA, $2A_{\mathcal{B}}$ with it. Zero coefficients, such as the sqrt zero mode, may still occupy recorded triplet slots. Physically nonzero matrix entries can be fewer, especially at $D=0$. The estimator uses the simpler bound above.

A complex CSR matrix with Z stored entries uses approximately

$$16Z + b_{\text{idx}}Z + b_{\text{idx}}(d+1) \text{ bytes} \quad (138)$$

for its numerical arrays, where b_{idx} depends on the sparse index type. The engine additionally retains H_0 , V , its QuTiP Hamiltonian, and transition rows/columns/values/mode IDs. Python lists and COO/CSR conversion add temporary peaks. Basis tuples, dictionary entries and photon-number arrays have at least MB occupation entries plus object overhead. These costs are excluded from the returned vector estimate.

feasible is only the heuristic $d \leq 100000$ and $G_{\text{vectors}} \leq 1$ GiB. It is not a memory guarantee, a physical accuracy criterion or an automatic engine limit. Notebooks check it before allocation. Estimates concern one trajectory: retained objects at several caps/schemes add their costs. Cap convergence keeps all its runs, even though they are final-only. Sweep peaks include a comparator and candidate together. The largest individual estimate is therefore not a total collection estimate.

Implicit fidelity keys examine $O(M_A B_A + M_B B_B)$ occupation entries. Sorting each key adds work dependent on its occupied-mode count; matching physical modes searches the current union for each second-grid mode. Keys and dictionaries add memory, but no ambient full vector is allocated. Choosing full for an actual propagation still incurs its exponential dimension.

2.3 Numbered code and comments on individual operations

Listings below are loaded directly from executed sources. Commentary tables identify current line ranges and relate their operations to the equations. The generator finds ordered engine markers and syntax-tree function ranges for experiments; line numbers are not copied from old versions. Comments cover validations, tensor ordering, phases, missing transitions, normalization, output shapes and statistical choices. Core-engine ranges are deliberately narrower than experiment function ranges. To reproduce the documentation from the repository root:

```
1 python documentation/build_code_notes.py
2 latexmk -pdf -cd documentation/main.tex
```

These commands generate documentation only.

```
src/xp_config.py
1 """Shared configuration for scripts and notebooks."""
2
3 from dataclasses import dataclass, replace
4
5
6 @dataclass
7 class ExperimentConfig:
8     param_photon: dict
9     param_atom: dict
10    param_time_evol: dict
11    cutoffs: dict | None = None
12    n_max: int = 3
13    truncation: str = "full+totalcap"
14    RWA: bool = False
15    store_state: bool = True
16    CTRL_M_EXPLICIT: bool = False
17    M: int | None = None
18    mode_selection: bool = False
19    photon_window: float = 1.5
20    atom_window: float = 0.25
21
22    def with_D(self, D):
23        """Copy the atom dictionary when changing the physical coupling."""
24        return replace(self, param_atom={**self.param_atom, "D": D})
```

Lines	Mathematical and implementation commentary
1–5	This module defines configuration data only. Imports do not allocate a grid or launch propagation.
6–7	The dataclass preserves named inputs and supports <code>dataclasses.replace</code> ; validation occurs in the consuming numerical routines.
8–8	Packet inputs k_0, σ_k, x_0 , preparation type, and either integer n or complex α ; see (15)–(22).
9–9	$\omega_0, D, L, x_{\text{tls}}$, coupling profile, and optional initial TLS label. The box length is stored here but also controls the photon grid.
10–10	Final time T , output spacing dt , and optional integration method/tolerances. Output spacing does not fix internal solver steps.
11–11	The radial band is required only under cutoff control. Explicit-count control ignores this dictionary.

Lines	Mathematical and implementation commentary
12–15	N and the basis select the retained occupation set, (11). RWA selects Hamiltonian terms; storage selects retained outputs, not a different Hilbert space.
16–17	True selects exact odd M with zero, (2); false selects the radial band, (3). No hidden decrement of M occurs.
18–21	Additional mode windows, applied after resolving the base grid. Their radii are factors times σ_k , (7).
22–24	Return a replaced configuration with a new atomic dictionary. Other inputs are reused; the supplied dictionary is not overwritten during a sweep.

src/grid.py

```

1 """One momentum-grid resolver for evolution and resource estimation."""
2
3 from numbers import Integral
4
5 import numpy as np
6
7
8 def integer(value, name, minimum=0):
9     if isinstance(value, bool) or not isinstance(value, Integral) or value < minimum:
10         raise ValueError(f"{name} must be an integer >= {minimum}")
11     return int(value)
12
13
14 def momentum_modes(param_atom, cutoffs=None, CTRL_M_EXPLICIT=False, M=None):
15     """Sorted signed momenta; no automatic zero-mode removal."""
16     L = float(param_atom["L"])
17     if not np.isfinite(L) or L <= 0:
18         raise ValueError("L must be finite and positive")
19     if not isinstance(CTRL_M_EXPLICIT, bool):
20         raise ValueError("CTRL_M_EXPLICIT must be a boolean")
21     spacing = 2 * np.pi / L
22     if CTRL_M_EXPLICIT:
23         M = integer(M, "M", minimum=1)
24         if M % 2 == 0:
25             raise ValueError("M must be odd for a symmetric grid containing k=0")
26         half = (M - 1) // 2
27         indices = np.arange(-half, half + 1)
28     else:
29         if cutoffs is None:
30             raise ValueError("IR/UV cutoffs are required when CTRL_M_EXPLICIT=False")
31         ir, uv = float(cutoffs["ir_cutoff"]), float(cutoffs["uv_cutoff"])
32         if not np.isfinite(ir) or not np.isfinite(uv) or not 0 <= ir <= uv:
33             raise ValueError("Cutoffs must be finite and satisfy 0 <= ir_cutoff <= uv_cutoff")
34         lower = int(np.ceil(ir / spacing - 1e-12))
35         upper = int(np.floor(uv / spacing + 1e-12))
36         positive = np.arange(max(1, lower), upper + 1)
37         zero = np.array([0], dtype=int) if ir == 0 else np.array([], dtype=int)
38         indices = np.concatenate((-positive[::-1], zero, positive))
39     modes = spacing * indices
40     if len(modes) == 0:
41         raise ValueError("The momentum band contains no modes")
42     return modes
43
44
45 def select_modes(k_tab, param_photon, param_atom, photon_window, atom_window):
46     """Packet/resonance windows, with a nearest-mode fallback per window."""
47     sigma = float(param_photon["sigma_k"])
48     if not np.isfinite(sigma) or sigma <= 0:
49         raise ValueError("sigma_k must be finite and positive")
50     if any(not np.isfinite(w) or w < 0 for w in (photon_window, atom_window)):
51         raise ValueError("Selection windows must be finite and nonnegative")
52     centres = ((param_photon["k_0"], photon_window * sigma),
53               (param_atom["omega_0"], atom_window * sigma),
54               (-param_atom["omega_0"], atom_window * sigma))
55     mask = np.zeros(len(k_tab), dtype=bool)
56     for centre, radius in centres:
57         window = np.abs(k_tab - centre) <= radius + 1e-12
58         if not window.any():
59             window[np.argmin(np.abs(k_tab - centre))] = True

```

```

60     mask |= window
61     return mask
62
63
64 def resolve_grid(config):
65     base = momentum_modes(config.param_atom, config.cutoffs, config.CTRL_M_EXPLICIT, config.M)
66     if config.mode_selection:
67         mask = select_modes(base, config.param_photon, config.param_atom,
68                             config.photon_window, config.atom_window)
69     return base, base[mask]
70     return base, base.copy()
71
72
73 def grid_summary(base, selected, L):
74     """Effective cutoffs and exact representability under either control."""
75     def describe(k_tab):
76         indices = np rint(k_tab / (2 * np.pi / L)).astype(int)
77         lower, upper = int(np.abs(indices).min()), int(np.abs(indices).max())
78         expected = {n for n in range(-upper, upper + 1) if abs(n) >= lower}
79         radial_exact = set(indices) == expected
80         nonzero = np.abs(k_tab[k_tab != 0])
81         return {"n_modes": len(k_tab), "k_min": float(k_tab.min()),
82                "k_max": float(k_tab.max()), "ir_effective": float(np.abs(k_tab).min()),
83                "uv_effective": float(np.abs(k_tab).max()),
84                "smallest_nonzero_k": float(nonzero.min()) if len(nonzero) else None,
85                "zero_present": bool(np.any(k_tab == 0)),
86                "radial_cutoffs_exact": radial_exact,
87                "equivalent_M": len(k_tab) if lower == 0 and radial_exact else None,
88                "integer_indices": indices.tolist()}
89     return {"delta_k": 2 * np.pi / L, "base": describe(base), "selected": describe(selected)}

```

Lines	Mathematical and implementation commentary
1–13	<code>integer</code> accepts integral types above its minimum, including NumPy integers. It rejects booleans and all floats rather than silently coercing them.
14–15	One grid generator for both evolution and estimation; its inputs describe the base grid, before any optional windows.
16–20	Convert and require finite $L > 0$. Require an actual boolean control so a truthy string cannot silently select the wrong branch.
21–21	Compute the physical spacing $\Delta k = 2\pi/L$, (1); it is never inferred from a chosen number of modes.
22–28	Require positive odd integer M and construct indices $-J, \dots, J$, $J = (M-1)/2$. Even counts raise an error before basis allocation.
29–33	Cutoff control requires finite ordered radial bounds $0 \leq \text{IR} \leq \text{UV}$. These validation steps are skipped when M controls the grid.
34–35	Ceiling/floor of cutoff ratios with dimensionless 10^{-12} boundary tolerances. The positive lower index is subsequently restricted to at least one; compare (5).
36–38	Enumerate positive indices, reflect them in reversed order for the negative side, and add zero only when input IR is exactly zero. A tiny positive IR is not rounded to zero.
39–44	Multiply integer indices by Δk and reject an empty band. Sorted signed momenta, not frequencies, are returned.
45–51	Validate positive finite σ_k and nonnegative finite factors. This is a requested subset operation; it is not a right-moving preparation filter.
52–54	Window centres $k_0, +\omega_0, -\omega_0$ with radii $w_p\sigma_k, w_a\sigma_k$, as in (7).
55–63	For each window use an absolute momentum tolerance, add the nearest mode if empty, and union its mask. Ties use the first sorted entry. The returned mask preserves base-grid ordering.
64–72	Resolve the same base grid for estimator and engine. With selection return its indexed subset; otherwise return a distinct array copy with the same values.

Lines	Mathematical and implementation commentary
73–77	Recover integer indices from the physical spacing and compute radial extrema. This representation report is independent of the photon cap and basis type.
78–80	Test equality with the entire symmetric radial band, (131); minima/maxima alone would miss holes or unpaired endpoints. Compute the smallest nonzero $ k $ separately.
81–88	Report count, signed extrema, effective cutoffs, zero, integer indices and exact representability. An equivalent explicit M exists only for a radial-exact set containing zero.
89–89	Return separate base and selected descriptions with their common physical spacing. A selected count may differ from the explicit base-grid input.

src/states.py

```

1  """Fock bases and Gaussian preparations; TLS is the last binary index."""
2
3  from itertools import combinations_with_replacement, product
4  from math import factorial
5
6  import numpy as np
7  import qutip
8
9  from .grid import integer
10
11
12  class FockBasis:
13      def __init__(self, modes, cap, truncation):
14          self.modes = modes = integer(modes, "modes", minimum=1)
15          self.cap = cap = integer(cap, "n_max")
16          self.truncation = truncation
17          vacuum = (0,) * modes
18          if truncation == "truncated":
19              field_states = [vacuum]
20              for m in range(modes):
21                  for n in range(1, cap + 1):
22                      occupation = list(vacuum)
23                      occupation[m] = n
24                      field_states.append(tuple(occupation))
25          elif truncation == "full+totalcap":
26              field_states = []
27              for total in range(cap + 1):
28                  for occupied in combinations_with_replacement(range(modes), total):
29                      field_states.append(tuple(np.bincount(occupied, minlength=modes)))
30          elif truncation == "full":
31              field_states = list(product(range(cap + 1), repeat=modes))
32          else:
33              raise ValueError(f"Unknown truncation: {truncation}")
34          self.states = field_states
35          self.index = {occupation: i for i, occupation in enumerate(field_states)}
36          self.photon_numbers = np.array([sum(n) for n in field_states])
37          self.dim = 2 * len(field_states)
38
39
40  def initial_state(basis, k_tab, param_photon, param_atom):
41      """Project a preparation onto the basis and normalize it once."""
42      k0, sigma = float(param_photon["k_0"]), float(param_photon["sigma_k"])
43      x0 = float(param_photon["x_0"])
44      if not np.isfinite([k0, sigma, x0]).all() or sigma <= 0:
45          raise ValueError("k_0 and x_0 must be finite; sigma_k must be finite and positive")
46      k_tab = np.asarray(k_tab, dtype=float)
47      exponent = -(k_tab - k0) ** 2 / (4 * sigma ** 2)
48      packet = np.exp(exponent - exponent.max()) * np.exp(-1j * k_tab * x0)
49      packet /= np.linalg.norm(packet)
50      atom = param_atom.get("initial_state", "g")
51      atom_coefs = {"g": (1, 0), "e": (0, 1),
52                  "+": (1 / np.sqrt(2), 1 / np.sqrt(2)),
53                  "-": (1 / np.sqrt(2), -1 / np.sqrt(2))}
54      if atom not in atom_coefs:
55          raise ValueError("initial_state must be g, e, +, or -")
56      atom_vector = np.asarray(atom_coefs[atom], dtype=complex)

```

```

57 vector = np.zeros(basis.dim, dtype=complex)
58 kind = param_photon.get("state", "number")
59 if kind == "number":
60     number = integer(param_photon.get("n", 1), "n")
61     if number > basis.cap:
62         raise ValueError("n must satisfy 0 <= n <= n_max")
63     if number == 0:
64         i = basis.index[(0,) * basis.modes]
65         vector[2 * i:2 * i + 2] = atom_vector
66     else:
67         for m, amplitude in enumerate(packet):
68             occupation = [0] * basis.modes
69             occupation[m] = number
70             i = basis.index[tuple(occupation)]
71             vector[2 * i:2 * i + 2] = amplitude * atom_vector
72 elif kind == "coherent":
73     alpha = complex(param_photon.get("alpha", 1.0))
74     if not np.isfinite(alpha):
75         raise ValueError("alpha must be finite")
76     for i, occupation in enumerate(basis.states):
77         coefficient = 1.0 + 0j
78         for m, n in enumerate(occupation):
79             coefficient *= (alpha * packet[m]) ** n / np.sqrt(float(factorial(n)))
80         vector[2 * i:2 * i + 2] = coefficient * atom_vector
81 else:
82     raise ValueError("state must be number or coherent")
83 norm = np.linalg.norm(vector)
84 if not np.isfinite(norm) or norm == 0:
85     raise ValueError("The projected initial state cannot be normalized")
86 return qutip.Qobj(vector / norm)

```

Lines	Mathematical and implementation commentary
1–11	Combinations with replacement enumerate total-cap occupations; a Cartesian product enumerates full. These algorithms avoid testing every candidate product against a total cap.
12–16	Validate at least one retained mode and nonnegative integer cap. Store the representation name; $N = 0$ is a valid vacuum-only photon space.
17–17	One zero tuple of length M represents the photon vacuum. TLS components are attached later through the alternating index rule (14).
18–24	Append $n e_m$ in mode-first, occupation-second order after vacuum. Multimode configurations are absent at every cap; dimension $2(1 + MN)$.
25–29	For each total from zero to N , enumerate multisets of occupied mode indices. <code>bincount</code> produces the tuple without duplicates; dimension $2^{\binom{M+N}{N}}$.
30–33	Cartesian product of $0, \dots, N$ on each mode. The final occupation changes fastest. Unknown names, including the removed extra-oscillator scheme, raise an error.
34–39	Store the exact tuples, their dictionary indices and photon totals. Ket dimension is twice the configuration count because the TLS remains a separate binary factor.
40–46	Read finite signed k_0 , finite x_0 and positive finite σ_k . All values in the supplied selected momentum array enter preparation.
47–49	Gaussian amplitude and phase from (15). Subtract the largest real exponent, then normalize the whole packet; the common stabilizing factor cancels.
50–55	Define the normalized $g, e, +, -$ vectors in TLS order (g, e) and reject unknown initial labels. Ground is the default.
56–58	Allocate the joint complex ket and select number or coherent preparation. The default is number, with $n = 1$ below.
59–66	Require integer $0 \leq n \leq N$. For $n = 0$ write the TLS vector onto vacuum exactly once, independent of how many modes exist.

Lines	Mathematical and implementation commentary
67–71	For $n > 0$, place $c_m a_0\rangle$ at occupation ne_m in alternating components. This is (18), not the collective-mode Fock state (19).
72–75	Read and validate complex α . The algorithm projects a product of coherent states, not a single coherent state restricted to one ray.
76–82	Multiply $(\alpha c_m)^{n_m} / \sqrt{n_m!}$ over all modes for each allowed tuple, then multiply both TLS components. The common coherent exponential is omitted because it cancels in (22).
83–86	Normalize the complete projected ket once, rejecting zero or nonfinite norm. The retained coherent mass and its basis dependence are derived in (25); no intersection normalization is involved.

src/hamiltonians.py

```

1 """Projected H = H0 + D V, with u_m = i f_m exp(-i k_m x_tls) / sqrt(L)."""
2
3 import numpy as np
4 import qutip
5 from scipy.sparse import coo_matrix, diags
6
7
8 class Hamiltonian:
9     def __init__(self, basis, k_tab, param_atom, RWA=False):
10         self.basis = basis
11         self.k_tab = np.asarray(k_tab)
12         self.param_atom = param_atom
13         self.RWA = RWA
14         L, omega, x = (float(param_atom[key]) for key in ("L", "omega_0", "x_tls"))
15         if not np.isfinite([L, omega, x]).all() or L <= 0 or omega < 0:
16             raise ValueError("L must be positive, omega_0 nonnegative, and x_tls finite")
17         self.H0 = self.free()
18         self.V = self.interaction()
19
20     def free(self):
21         energies = np.zeros(self.basis.dim)
22         for i, occupation in enumerate(self.basis.states):
23             field_energy = np.dot(np.abs(self.k_tab), occupation)
24             energies[2 * i] = field_energy
25             energies[2 * i + 1] = field_energy + self.param_atom["omega_0"]
26         return diags(energies, format="csr")
27
28     def interaction(self):
29         profile = self.param_atom["coupling"]
30         if profile == "sqrt":
31             form_factor = np.sqrt(np.abs(self.k_tab))
32         elif profile == "flat":
33             form_factor = np.ones(self.basis.modes)
34         else:
35             raise ValueError("coupling must be flat or sqrt")
36         u = 1j * form_factor * np.exp(-1j * self.k_tab * self.param_atom["x_tls"])
37         u /= np.sqrt(self.param_atom["L"])
38         rows, cols, values, mode_ids = [], [], [], []
39         for i, occupation in enumerate(self.basis.states):
40             for atom in (0, 1):
41                 col = 2 * i + atom
42                 for m in range(self.basis.modes):
43                     if occupation[m] and (not self.RWA or atom == 0):
44                         target = list(occupation)
45                         target[m] -= 1
46                         j = self.basis.index[tuple(target)]
47                         rows.append(2 * j + 1 - atom)
48                         cols.append(col)
49                         values.append(u[m] * np.sqrt(occupation[m]))
50                         mode_ids.append(m)
51                     if not self.RWA or atom == 1:
52                         target = list(occupation)
53                         target[m] += 1
54                         j = self.basis.index.get(tuple(target))
55                         if j is not None:
56                             rows.append(2 * j + 1 - atom)

```

```

57         cols.append(col)
58         values.append(u[m].conjugate() * np.sqrt(occupation[m] + 1))
59         mode_ids.append(m)
60     self.transition_rows = np.asarray(rows, dtype=int)
61     self.transition_cols = np.asarray(cols, dtype=int)
62     self.transition_values = np.asarray(values, dtype=complex)
63     self.transition_modes = np.asarray(mode_ids, dtype=int)
64     return coo_matrix((values, (rows, cols)), shape=(self.basis.dim, self.basis.dim)).tocsr()
65
66     def build_hamiltonian(self, D=None):
67         D = self.param_atom["D"] if D is None else D
68         if np.iscomplexobj(D) or not np.isfinite(D):
69             raise ValueError("D must be finite and real")
70         return qutip.Qobj(self.H0 + float(D) * self.V)

```

Lines	Mathematical and implementation commentary
1–7	Use COO triplets and CSR storage rather than a dense $d \times d$ allocation. QuTiP wraps the final sparse operator for evolution.
8–19	Store basis and physical parameters; require finite $L > 0$, $\omega_0 \geq 0$ and finite TLS position. Build the free term and interaction separately.
20–27	For each occupation compute $\sum_m k_m n_m$, then place it on ground and add ω_0 on excited TLS. The ground-state energy origin is explicit, (33).
28–35	Select $f_m = \sqrt{ k_m }$ or 1; reject other labels. The signed momentum survives in the spatial phase even though the profile uses $ k $.
36–37	Compute $u_m = i f_m e^{-i k_m x_{\text{tls}}} / \sqrt{L}$, (30). D is not multiplied yet, allowing the same interaction matrix to be scaled.
38–38	Allocate row/column/value lists and a mode tag per transition. Tags support exact interaction-energy attribution in (83).
39–42	Iterate occupations, TLS labels and changed modes. The matrix column is the source $2i + s$; the target row flips the TLS to $1 - s$.
43–48	Annihilation requires positive occupation. Under RWA it also requires source TLS g . All retained bases are downward closed, so dictionary lookup of the decremented tuple is valid.
49–50	Record $u_m \sqrt{n_m}$ and the changed-mode tag. The factor is the bosonic matrix element, with the imposed complex annihilation phase.
51–54	Creation is allowed for either TLS without RWA and only source e under RWA. The incremented tuple is tested, not assumed present.
55–59	Omit targets outside the chosen space, exactly implementing projection. Retained creation receives $u_m^* \sqrt{n_m + 1}$, conjugate to its reverse annihilation.
60–65	Keep numerical triplet/tag arrays for diagnostics and convert COO to CSR. Zero-valued triplets can remain recorded, notably for sqrt at $k = 0$.
66–70	Require finite real D and return $H_0 + DV$ as a QuTiP operator. Hermiticity follows from paired transitions and real scaling, not solver output normalization.

```

src/fidelities.py
1 """Exact squared state/TLS fidelities in an implicit common full basis."""
2
3 import numpy as np
4 import qutip
5
6
7 def _vector(ket, basis):
8     vector = ket.full()[ :, 0] if isinstance(ket, qutip.Qobj) else np.asarray(ket)
9     if vector.shape != (basis.dim,):
10         raise ValueError("Ket dimension does not match the Fock basis")
11     norm = np.linalg.norm(vector)
12     if not np.isfinite(norm) or norm == 0:
13         raise ValueError("A fidelity requires nonzero finite kets")

```

```

14     return vector / norm
15
16
17 def atom_density_matrix(vector):
18     coefficients = np.asarray(vector).reshape(-1, 2)
19     return coefficients.T @ coefficients.conj()
20
21
22 def _mode_maps(k_a, k_b):
23     """Align physical modes, allowing roundoff in equivalent grids."""
24     union = list(np.asarray(k_a, dtype=float))
25     map_a = list(range(len(union)))
26     map_b = []
27     for k in k_b:
28         matches = np.flatnonzero(np.isclose(union, k, atol=1e-12, rtol=0))
29         if len(matches) > 1:
30             raise ValueError("Ambiguous physical momentum alignment")
31         if len(matches):
32             map_b.append(int(matches[0]))
33         else:
34             map_b.append(len(union))
35             union.append(float(k))
36     return map_a, map_b
37
38
39 def _components(basis, vector, mapping):
40     amplitudes = {}
41     for i, occupation in enumerate(basis.states):
42         key = tuple(sorted((mapping[m], int(n)) for m, n in enumerate(occupation) if n))
43         amplitudes[key] = vector[2 * i:2 * i + 2]
44     return amplitudes
45
46
47 def compare_states(ket_a, basis_a, k_a, ket_b, basis_b, k_b):
48     """Isometric embedding without allocating the ambient full-space vectors."""
49     a, b = _vector(ket_a, basis_a), _vector(ket_b, basis_b)
50     map_a, map_b = _mode_maps(k_a, k_b)
51     components_a = _components(basis_a, a, map_a)
52     components_b = _components(basis_b, b, map_b)
53     overlap = sum(np.vdot(components_a[key], components_b[key])
54                  for key in components_a.keys() & components_b.keys())
55     rho_a, rho_b = qutip.Qobj(atom_density_matrix(a)), qutip.Qobj(atom_density_matrix(b))
56     return {"F_state": float(np.clip(abs(overlap) ** 2, 0, 1)),
57           "F_atom": float(np.clip(qutip.fidelity(rho_a, rho_b) ** 2, 0, 1))}
58
59
60 def fidelities_over_time(experiment_a, experiment_b):
61     if not experiment_a.store_state or not experiment_b.store_state:
62         raise ValueError("Both experiments must store states for a time series")
63     if not np.array_equal(experiment_a.times, experiment_b.times):
64         raise ValueError("Experiments must use the same output times")
65     if not np.isclose(experiment_a.param_atom["L"], experiment_b.param_atom["L"],
66                      atol=1e-12, rtol=0):
67         raise ValueError("Experiments must use the same box length for physical-mode embedding")
68     pairs = zip(experiment_a.result.states, experiment_b.result.states)
69     values = [compare_states(a, experiment_a.basis, experiment_a.k_tab,
70                             b, experiment_b.basis, experiment_b.k_tab) for a, b in pairs]
71     return {name: np.array([value[name] for value in values])
72           for name in ("F_state", "F_atom")}

```

Lines	Mathematical and implementation commentary
1–6	This module implements exactly two squared metrics; ambient full-space vectors and a ratio field proxy are not constructed.
7–16	Accept a QuTiP ket or one-dimensional array of the basis dimension. Normalize the entire nonzero finite ket for metric evaluation, retaining raw norm diagnostics elsewhere.
17–21	Reshape alternating components to C with shape $(B, 2)$, then compute $C^T C^*$ as in (40). For a raw ket its trace is the squared norm.

Lines	Mathematical and implementation commentary
22–26	Start the union with the first physical momentum list and its identity map. Internal union order need not be sorted because occupation keys are canonicalized.
27–38	Match each second-grid mode with absolute tolerance 10^{-12} and zero relative tolerance. Reuse its union index, append if absent, or reject an ambiguous match.
39–46	For each tuple, discard zero occupations and sort positive (union index, n) pairs. Store the two TLS amplitudes; missing modes are vacuum implicitly and vacuum has an empty key.
47–52	Normalize both kets, align physical modes, and build occupation dictionaries. This low-level API cannot check box length or Hamiltonian conventions, which remain caller responsibilities.
53–54	Sum the conjugate first-state times second-state TLS dot product over every shared physical key, (121). Do not normalize on that intersection; example (122) would otherwise be wrong.
55–59	Reduce the individually normalized kets, square QuTiP root fidelity, and return the two metrics clipped for numerical overshoot. Vacuum embedding leaves these atomic reductions unchanged.
60–67	Require both histories, exactly matching output arrays and equal L to absolute 10^{-12} . These checks precede pairing, preventing silent comparison of different time samples or boxes.
68–72	Compare matching stored states and return two one-dimensional arrays of length N_t . The simulated comparator is an independent input, not the ambient embedding choice.

src/energy_profile.py

```

1 """Exact energy partitions by field mode and total photon occupation."""
2
3 import numpy as np
4
5
6 class EnergyProfile:
7     def __init__(self, experiment):
8         self.experiment = experiment
9         self.occupations = np.asarray(experiment.basis.states)
10        self.excitation_axis = np.arange(experiment.photon_numbers.max() + 1)
11
12    def energy_modes_vec(self, vector):
13        exp = self.experiment
14        probability = np.abs(vector.reshape(-1, 2)) ** 2
15        field = np.abs(exp.k_tab) * (self.occupations.T @ probability.sum(axis=1))
16        h = exp.hamiltonian
17        contributions = np.real(vector[h.transition_rows].conj() *
18                                exp.param_atom["D"] * h.transition_values *
19                                vector[h.transition_cols])
20        interaction = np.bincount(h.transition_modes, weights=contributions,
21                                minlength=exp.n_modes)
22        atom = exp.param_atom["omega_0"] * probability[:, 1].sum()
23        return field + interaction, float(atom)
24
25    def energy_excitations_vec(self, vector):
26        exp = self.experiment
27        energy = np.real(vector.conj() * (exp.hamiltonian.H0 + exp.param_atom["D"] *
28                                        exp.hamiltonian.V).dot(vector))
29        return np.bincount(exp.photon_numbers, weights=energy.reshape(-1, 2).sum(axis=1),
30                            minlength=len(self.excitation_axis))

```

Lines	Mathematical and implementation commentary
1–5	Energy partitions consume the already-built Hamiltonian and propagated vector; there is no second interaction implementation.
6–11	Cache the occupation matrix $n_{i,m}$ and photon-total axis $0, \dots, \nu_{\max}$. The function $\nu(q)$ and projectors Q_ν are defined in (91); full can reach MN .
12–14	Reshape the raw vector to $C_{i,s}$ of shape $(B, 2)$ and form $ C_{i,s} ^2$. Calculations use raw quadratic expectations; see (115) for normalization.
15–16	The transpose occupation matrix times summed TLS probabilities gives $\langle N_m \rangle = \sum_{i,s} n_{i,m} C_{i,s} ^2$. Multiply by $\omega_m = k_m $ to obtain the bare field term F_m in (74).
17–19	For each record ℓ , index source c_ℓ , target r_ℓ and complex value z_ℓ , then compute $\Re[v_{r_\ell}^* D z_\ell v_{c_\ell}]$. The record and derivation are (78)–(83); values contain u_m but not D .
20–21	Weighted <code>bincount</code> by tag m_ℓ sums directed terms into $W_m = \langle DV_m \rangle$. Both reverse orientations are already present; do not multiply this accumulated result by two, (85).
22–24	Return $E_m = F_m + W_m$ and the separate bare TLS term $E_a = \omega_0 \sum_i C_{i,1} ^2$. Interaction is assigned to the mode columns by convention, not included in the TLS marker, (76).
25–26	Use the same experiment and Hamiltonian; the result is indexed by photon total, keeping both atomic components. It is independent of the mode-tag partition.
27–28	Form the sparse matrix action and its row terms $\Re[v_q^*(Hv)_q]$, which define (95). These terms include cross-sector coherences rather than projecting the Hamiltonian to its diagonal blocks.
29–30	First sum the two TLS rows of each photon configuration, then group by total photon number. This gives $E_\nu = \langle (Q_\nu H + H Q_\nu)/2 \rangle$, (99); it is not conditional energy (106).

src/experiment.py

```

1 """Pure-state dynamics and diagnostics shared by every experiment."""
2
3 from math import comb
4
5 import numpy as np
6 import qutip
7
8 from .energy_profile import EnergyProfile
9 from .fidelities import atom_density_matrix
10 from .grid import grid_summary, integer, momentum_modes, resolve_grid
11 from .hamiltonians import Hamiltonian
12 from .states import FockBasis, initial_state
13 from .xp_config import ExperimentConfig
14
15
16 def output_times(param_time_evol):
17     T, dt = float(param_time_evol["T"]), float(param_time_evol["dt"])
18     if not np.isfinite([T, dt]).all() or T <= 0 or dt <= 0:
19         raise ValueError("T and dt must be finite and positive")
20     count = int(np.floor(T / dt + 1e-12))
21     if count == 0:
22         return np.array([0.0, T])
23     times = np.arange(count + 1) * dt
24     if times[-1] < T and not np.isclose(times[-1], T, atol=1e-12, rtol=0):
25         times = np.append(times, T)
26     else:
27         times[-1] = T
28     return times
29
30
31 def resource_estimation(param_atom, param_time_evol, cutoffs=None,
32                         n_max=3, truncation="full+totalcap", store_state=True,

```

```

33         CTRL_M_EXPLICIT=False, M=None, param_photon=None,
34         mode_selection=False, photon_window=1.5, atom_window=0.25,
35         print_report=True):
36     config = ExperimentConfig(param_photon or {}, param_atom, param_time_evol, cutoffs,
37                               n_max, truncation, store_state=store_state,
38                               CTRL_M_EXPLICIT=CTRL_M_EXPLICIT, M=M,
39                               mode_selection=mode_selection, photon_window=photon_window,
40                               atom_window=atom_window)
41     base, selected = resolve_grid(config)
42     modes, cap = len(selected), integer(n_max, "n_max")
43     if truncation == "full+totalcap":
44         dimension = 2 * comb(modes + cap, cap)
45     elif truncation == "truncated":
46         dimension = 2 * (1 + modes * cap)
47     elif truncation == "full":
48         dimension = 2 * (cap + 1) ** modes
49     else:
50         raise ValueError(f"Unknown truncation: {truncation}")
51     times = output_times(param_time_evol)
52     ket_gib = dimension * 16 / 2**30
53     history_gib = ket_gib * (len(times) if store_state else 1)
54     retained_gib = 2 * history_gib
55     nnz_bound = dimension * (1 + 2 * modes)
56     summary = grid_summary(base, selected, param_atom["L"])
57     feasible = dimension <= 100_000 and retained_gib <= 1.0
58     estimate = {"n_modes": modes, "dimension": dimension, "n_times": len(times),
59                "ket_gib": ket_gib, "history_gib": history_gib,
60                "retained_vectors_gib": retained_gib, "hamiltonian_nnz_bound": nnz_bound,
61                "feasible": feasible, "grid": summary}
62     if print_report:
63         control = f"explicit M={M}" if CTRL_M_EXPLICIT else f"cutoffs={cutoffs}"
64         print(f"Grid control: {control}; L={param_atom['L']:g}; delta_k={summary['delta_k']:.8g}")
65         for label in ("base", "selected"):
66             item = summary[label]
67             print(f" {label}: M={item['n_modes']}; k in [{item['k_min']:.8g}, {item['k_max']:.8g}]; "
68                   f"effective (IR,UV)={({item['ir_effective']:.8g},{item['uv_effective']:.8g}); "
69                   f"zero={item['zero_present']})"
70                   print(f" radial cutoffs exact={item['radial_cutoffs_exact']}; "
71                         f"equivalent explicit M={item['equivalent_M']})"
72             print(f"Basis: {truncation}; dimension={dimension:}; output times={len(times)}")
73             print(f"Ket={ket_gib:.4g} GiB; solver history={history_gib:.4g} GiB; "
74                   f"retained vectors approximately={retained_gib:.4g} GiB")
75             print(f"Hamiltonian nnz <= {nnz_bound:}; heuristic feasible={feasible}")
76             print("Memory excludes sparse matrices, basis/Python objects, solver workspace and plots.")
77     return estimate
78
79
80 def estimate_config(config, print_report=True):
81     return resource_estimation(config.param_atom, config.param_time_evol, config.cutoffs,
82                               config.n_max, config.truncation, config.store_state,
83                               config.CTRL_M_EXPLICIT, config.M, config.param_photon,
84                               config.mode_selection, config.photon_window, config.atom_window,
85                               print_report)
86
87
88 class Experiment:
89     def __init__(self, config):
90         self.config = config
91         self.param_photon, self.param_atom = dict(config.param_photon), dict(config.param_atom)
92         self.param_time_evol = dict(config.param_time_evol)
93         self.n_max, self.truncation = config.n_max, config.truncation
94         self.RWA, self.store_state = config.RWA, config.store_state
95         self.base_k_tab, self.k_tab = resolve_grid(config)
96         self.n_modes = len(self.k_tab)
97         self.times = output_times(self.param_time_evol)
98         self.basis = FockBasis(self.n_modes, self.n_max, self.truncation)
99         self.photon_numbers = self.basis.photon_numbers
100        self.hamiltonian = Hamiltonian(self.basis, self.k_tab, self.param_atom, self.RWA)
101        self.H = self.hamiltonian.build_hamiltonian()
102        self.state0 = initial_state(self.basis, self.k_tab, self.param_photon, self.param_atom)
103        self.c_g_array = self.c_e_array = self.observables = None
104
105    def propagate_state(self, progress=False):
106        self.result = qutip.sesolve(self.H, self.state0, self.times, options={
107            "method": self.param_time_evol.get("method", "bdf"),

```

```

108         "store_states": self.store_state, "store_final_state": True,
109         "normalize_output": False, "rtol": self.param_time_evol.get("rtol", 1e-9),
110         "atol": self.param_time_evol.get("atol", 1e-11),
111         "progress_bar": "tqdm" if progress else False})
112     vectors = np.array([s.full()[:, 0] for s in self.result.states]) if self.store_state \
113         else self.result.final_state.full()[:, 0]
114     self.c_g_array, self.c_e_array = vectors[..., 0::2], vectors[..., 1::2]
115     return self.c_g_array, self.c_e_array
116
117 def _one_photon_indices(self):
118     indices = np.flatnonzero(self.photon_numbers == 1)
119     mode_ids = np.array([np.argmax(self.basis.states[i]) for i in indices], dtype=int)
120     return indices, mode_ids
121
122 def compute_observables(self):
123     if self.c_g_array is None:
124         raise RuntimeError("Call propagate_state first")
125     ground, excited = np.atleast_2d(self.c_g_array), np.atleast_2d(self.c_e_array)
126     probability = np.abs(ground) ** 2 + np.abs(excited) ** 2
127     indices, modes = self._one_photon_indices()
128     self.observables = {"time": self.times if self.store_state else self.times[-1:],
129                         "norm": probability.sum(axis=1),
130                         "p_excited": (np.abs(excited) ** 2).sum(axis=1)}
131     for name, mask in (("p_transmitted", self.k_tab[modes] > 0),
132                       ("p_reflected", self.k_tab[modes] < 0),
133                       ("p_zero_1g", self.k_tab[modes] == 0)):
134         self.observables[name] = (np.abs(ground[:, indices[mask]]) ** 2).sum(axis=1)
135     for n in range(int(self.photon_numbers.max()) + 1):
136         self.observables[f"p_{n}_photon"] = probability[:, self.photon_numbers == n].sum(axis=1)
137     return self.observables
138
139 def one_photon_wavefunction(self, i, x_tab):
140     if self.c_g_array is None:
141         raise RuntimeError("Call propagate_state first")
142     coefficients = self.c_g_array[i] if self.store_state else self.c_g_array
143     indices, modes = self._one_photon_indices()
144     return np.exp(1j * np.outer(x_tab, self.k_tab[modes])) @ coefficients[indices] / \
145         np.sqrt(self.param_atom["L"])
146
147 def _vectors_for(self, t=None):
148     if self.c_g_array is None:
149         raise RuntimeError("Call propagate_state first")
150     if not self.store_state:
151         if t is not None and t != -1 and not np.isclose(t, self.times[-1]):
152             raise ValueError("Only the final state was stored")
153         return [self.result.final_state.full()[:, 0]]
154     if t is None:
155         return [s.full()[:, 0] for s in self.result.states]
156     if t != -1 and not 0 <= t <= self.times[-1]:
157         raise ValueError("Requested time lies outside the stored interval")
158     i = -1 if t == -1 else int(np.argmin(abs(self.times - t)))
159     return [self.result.states[i].full()[:, 0]]
160
161 def compute_atom_density_matrix(self, t=None):
162     matrices = [atom_density_matrix(v) for v in self._vectors_for(t)]
163     return matrices if t is None else matrices[0]
164
165 def compute_excited_probability(self, t=None):
166     values = np.array([atom_density_matrix(v)[1, 1].real for v in self._vectors_for(t)])
167     return values if t is None else float(values[0])
168
169 def compute_entropy(self, t=None):
170     values = np.array([qutip.entropy_vn(qutip.Qobj(atom_density_matrix(v) / np.vdot(v, v).real))
171                       for v in self._vectors_for(t)])
172     return values if t is None else float(values[0])
173
174 def compute_energy(self, t=None):
175     matrix = self.hamiltonian.H0 + self.param_atom["D"] * self.hamiltonian.V
176     values = np.array([np.vdot(v, matrix @ v).real for v in self._vectors_for(t)])
177     return values if t is None else float(values[0])
178
179 def compute_energy_profile_modes(self, t=None):
180     profile = EnergyProfile(self)
181     results = [profile.energy_modes_vec(v) for v in self._vectors_for(t)]
182     modes, atom = np.array([r[0] for r in results]), np.array([r[1] for r in results])

```

```

183     return self.k_tab, modes if t is None else modes[0], 0.0, atom if t is None else float(atom[0])
184
185 def compute_energy_profile_excitations(self, t=None):
186     profile = EnergyProfile(self)
187     values = np.array([profile.energy_excitations_vec(v) for v in self._vectors_for(t)])
188     return profile.excitation_axis, values if t is None else values[0]

```

Lines	Mathematical and implementation commentary
1–15	One common engine imports grid, basis, Hamiltonian and diagnostics. <code>momentum_modes</code> remains importable from this module for API convenience.
16–30	Validate finite positive T, dt . Generate nominal output spacing with a roundoff tolerance, always including zero and exact T ; if $dt > T$, return both endpoints.
31–42	Build configuration from the dictionary API and call the same grid resolver as propagation. Optional selection requires packet information; dimension is evaluated without enumerating a basis.
43–50	Use exact dimensions (13) with the selected count. Reject unknown basis names; no dense allocation is needed to estimate exponential full-space growth.
51–55	Count outputs and estimate $16d$ bytes per ket, retained history depending on storage, two retained vector copies and the sparse-entry bound, (133).
56–79	Return base/selected representations and heuristic feasibility. Printed output exposes other-control implications and explicitly names excluded memory costs; it does not enforce a solver allocation limit.
80–87	Forward all grid, cap, selection and storage inputs from the dataclass to the dictionary estimator without a separate estimate formula.
88–104	Copy physical dictionaries, resolve base/selected grids, generate output times, build basis and projected Hamiltonian, then prepare the ket. Coefficient arrays are absent until propagation.
105–116	Run <code>sesolve</code> with BDF/default tolerances, raw output normalization and final-state storage. Build a contiguous vector array; alternating ground/excited slices are views, shape (N_t, B) or $(B,)$ for final-only.
117–121	Find tuples with total exactly one, then identify their occupied mode. This works across all three basis orderings and gives an empty list at $N = 0$.
122–138	Require propagation, promote final-only arrays internally, and compute raw norm, TLS excitation, signed $1g$ populations and all photon totals. Sum identities are (44).
139–146	The first argument is an output index, not a physical time. Extract only $1g$ amplitudes and multiply by positive Fourier phases and $L^{-1/2}$, (45).
147–160	Resolve diagnostic requests: all retained states for None, final for -1 , or nearest stored output for an in-range time. No interpolation occurs; final-only runs reject earlier requests.
161–164	Reduce each requested raw ket. None returns a list, an explicit time one matrix; final-only with None returns a one-element list.
165–168	Return the real excited diagonal of the raw TLS reduction. None returns an array, an explicit time a scalar; norm drift remains visible.
169–173	Normalize the atomic reduction by the full squared ket norm before natural-log von Neumann entropy. This is (41), bounded by $\log 2$ for normalized valid states.
174–178	Apply the same sparse $H_0 + DV$ and take the real raw expectation. Return a time array or scalar without introducing a second energy-origin convention.
179–184	For requested kets, return $(k, E_m, 0.0, E_a)$ with histories or one profile. The zero is a retained tuple-interface placeholder, not a supplemental atomic mode.

Lines	Mathematical and implementation commentary
185–188	Return the photon-total axis and row-grouped energy partition. The time selector follows the same rules as other diagnostics; summed energy agrees with direct expectation.

```

experiment/_common.py
1  """Shared experiment execution, CLI inputs and exports."""
2
3  import argparse
4  from dataclasses import asdict
5  import json
6  from pathlib import Path
7
8  import numpy as np
9
10 from src.experiment import Experiment, estimate_config
11 from src.xp_config import ExperimentConfig
12
13
14 def run(config, progress=False):
15     experiment = Experiment(config)
16     experiment.propagate_state(progress=progress)
17     experiment.compute_observables()
18     return experiment
19
20
21 def save_arrays(path, config, **arrays):
22     path = Path(path)
23     path.parent.mkdir(parents=True, exist_ok=True)
24     metadata = json.dumps(asdict(config), default=lambda value: str(value))
25     np.savez(path, configuration_json=metadata, **arrays)
26
27
28 def parser(description):
29     p = argparse.ArgumentParser(description=description)
30     p.add_argument('--D', type=float, default=0.2)
31     p.add_argument('--L', type=float, default=2 * np.pi)
32     p.add_argument('--omega-0', type=float, default=1.0)
33     p.add_argument('--x-tls', type=float, default=0.0)
34     p.add_argument('--coupling', choices=('sqrt', 'flat'), default='sqrt')
35     p.add_argument('--ctrl-m-explicit', action=argparse.BooleanOptionalAction, default=False)
36     p.add_argument('--M', type=int, default=5)
37     p.add_argument('--ir-cutoff', type=float, default=0.0)
38     p.add_argument('--uv-cutoff', type=float, default=2.0)
39     p.add_argument('--k-0', type=float, default=1.0)
40     p.add_argument('--sigma-k', type=float, default=0.4)
41     p.add_argument('--x-0', type=float, default=-2.0)
42     p.add_argument('--photon-state', choices=('number', 'coherent'), default='number')
43     p.add_argument('--n', type=int, default=1)
44     p.add_argument('--alpha', type=complex, default=1.0)
45     p.add_argument('--atom-state', choices=('g', 'e', '+', '-'), default='g')
46     p.add_argument('--n-max', type=int, default=2)
47     p.add_argument('--truncation', choices=('full', 'full+totalcap', 'truncated'), default='full+totalcap')
48     p.add_argument('--T', type=float, default=4.0)
49     p.add_argument('--dt', type=float, default=0.05)
50     p.add_argument('--RWA', action='store_true')
51     p.add_argument('--out', type=Path)
52     p.add_argument('--progress', action='store_true')
53     return p
54
55
56 def config_from_args(args):
57     return ExperimentConfig(
58         param_photon={'k_0': args.k_0, 'sigma_k': args.sigma_k, 'x_0': args.x_0,
59                      'state': args.photon_state, 'n': args.n, 'alpha': args.alpha},
60         param_atom={'omega_0': args.omega_0, 'D': args.D, 'L': args.L,
61                   'x_tls': args.x_tls, 'coupling': args.coupling, 'initial_state': args.atom_state},
62         param_time_evol={'T': args.T, 'dt': args.dt},
63         cutoffs={'ir_cutoff': args.ir_cutoff, 'uv_cutoff': args.uv_cutoff},
64         n_max=args.n_max, truncation=args.truncation, RWA=args.RWA,
65         CTRL_M_EXPLICIT=args.ctrl_m_explicit, M=args.M)
66

```

```

67
68 def finish(fig, args, config, **arrays):
69     import matplotlib.pyplot as plt
70     if args.out:
71         args.out.parent.mkdir(parents=True, exist_ok=True)
72         fig.savefig(args.out, dpi=150)
73         save_arrays(args.out.with_suffix('.npz'), config, **arrays)
74         plt.close(fig)
75         print(f'Saved {args.out}')
76     else:
77         plt.show()

```

Lines	Mathematical and implementation commentary
14–18	run : Construct one Experiment , propagate, then compute observables in that order. Return the numerical object so notebook analyses can reuse its kets and Hamiltonian.
21–25	save_arrays : Create the output directory and write NPZ arrays plus configuration_json from the base dataclass. Non-JSON values stringify; ordinary numerical arrays can be loaded without pickle.
28–53	parser : Shared CLI names use $D, L, M, k_0, \sigma_k, x_0, N, T$. Default profile is sqrt . Explicit M requires its boolean flag; default CLI grid control uses cutoffs.
56–65	config_from_args : Construct the three dictionaries and dataclass with the current conventions. Both count and cutoff inputs may be stored, but only the selected control is active; historical values are not automatically converted.
68–77	finish : With an output path, save the figure and same-stem NPZ, then close it. Otherwise display the figure. Backend selection remains the environment or notebook responsibility.

```

experiment/atom_evolution.py
1  """TLS population and entropy for several photon bases."""
2
3  from dataclasses import replace
4
5  from ._common import config_from_args, finish, parser, run
6
7
8  def run_atom_evolution(config, schemes=('full+totalcap', 'truncated'), progress=False):
9      return {scheme: run(replace(config, truncation=scheme, store_state=True), progress)
10             for scheme in schemes}
11
12
13  def plot_atom_evolution(runs):
14      import matplotlib.pyplot as plt
15      fig, axes = plt.subplots(1, 2, figsize=(10, 4))
16      for scheme, experiment in runs.items():
17          axes[0].plot(experiment.times, experiment.compute_excited_probability(), label=scheme)
18          axes[1].plot(experiment.times, experiment.compute_entropy(), label=scheme)
19          axes[0].set(xlabel='t', ylabel=r'$P_e$')
20          axes[1].set(xlabel='t', ylabel=r'$S_{\mathrm{TLS}}$')
21      for ax in axes:
22          ax.legend()
23      fig.tight_layout()
24      return fig
25
26
27  def main(argv=None):
28      p = parser(__doc__)
29      p.add_argument('--schemes', default='full+totalcap,truncated')
30      args = p.parse_args(argv)
31      config = config_from_args(args)
32      from src.experiment import estimate_config
33      for scheme in args.schemes.split(','):
34          estimate_config(replace(config, truncation=scheme))
35      runs = run_atom_evolution(config, args.schemes.split(','), args.progress)
36      arrays = {f'{scheme}_P_e': exp.compute_excited_probability() for scheme, exp in runs.items()}
37      finish(plot_atom_evolution(runs), args, config, time=next(iter(runs.values())).times, **arrays)

```

```

38
39
40 if __name__ == '__main__':
41     main()

```

Lines	Mathematical and implementation commentary
8–10	<code>run_atom_evolution</code> : Replace the configuration by each scheme and force history storage. Return a dictionary of experiments, so excitation, entropy and notebook fidelity derive from the same stored trajectories.
13–24	<code>plot_atom_evolution</code> : Plot raw P_e and normalized TLS entropy against output times for every returned scheme. This plot does not itself propagate or compute cross-state fidelity.
27–37	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

```

experiment/compare_mode_selection.py
1 """Truncation and mode-selection errors relative to one finite trajectory."""
2
3 from dataclasses import replace
4
5 import numpy as np
6
7 from src.fidelities import fidelities_over_time
8 from ._common import config_from_args, finish, parser, run
9
10
11 def run_mode_selection(config, D_values=(0.02, 0.1, 0.2),
12                       schemes=('truncated', 'full+totalcap'),
13                       photon_windows=(0.5, 1.5, 3.0), atom_windows=(0.0, 0.5, 1.5),
14                       D_heatmap=None, progress=False):
15     if not config.store_state:
16         raise ValueError('Time-mean comparisons require store_state=True')
17     D_values = np.asarray(D_values, dtype=float)
18     schemes = tuple(schemes)
19     pw, aw = np.asarray(photon_windows), np.asarray(atom_windows)
20     shape = (len(schemes), 2, len(D_values))
21     metrics = {name: np.empty(shape) for name in ('F_state', 'F_atom', 'F_state_initial', 'F_atom_initial')}
22     for i, D in enumerate(D_values):
23         current = config.with_D(float(D))
24         reference = run(replace(current, truncation='full+totalcap', mode_selection=False), progress)
25         for j, scheme in enumerate(schemes):
26             for selected in (False, True):
27                 candidate = run(replace(current, truncation=scheme, mode_selection=selected), progress)
28                 values = fidelities_over_time(candidate, reference)
29                 for name, series in values.items():
30                     metrics[name][j, int(selected), i] = np.mean(series)
31                 metrics[f'{name}_initial'][j, int(selected), i] = series[0]
32     current = config.with_D(config.param_atom['D'] if D_heatmap is None else D_heatmap)
33     reference = run(replace(current, truncation='full+totalcap', mode_selection=False), progress)
34     heatmaps = {name: np.empty((len(schemes), len(pw), len(aw))) for name in ('F_state', 'F_atom')}
35     for j, scheme in enumerate(schemes):
36         for x, photon_window in enumerate(pw):
37             for y, atom_window in enumerate(aw):
38                 candidate = run(replace(current, truncation=scheme, mode_selection=True,
39                                         photon_window=float(photon_window), atom_window=float(atom_window)),
40                               progress)
41                 for name, series in fidelities_over_time(candidate, reference).items():
42                     heatmaps[name][j, x, y] = np.mean(series)
43     return {'D': D_values, 'schemes': schemes, 'selection': np.array([False, True]),
44           'photon_windows': pw, 'atom_windows': aw, 'D_heatmap': current.param_atom['D'],
45           **metrics, **{f'{name}_heatmap': value for name, value in heatmaps.items()}}
46
47 def plot_mode_selection(results):
48     import matplotlib.pyplot as plt
49     schemes = results['schemes']
50     fig, axes = plt.subplots(len(schemes) + 1, 2, figsize=(11, 3 * (len(schemes) + 1)), squeeze=False)

```

```

51 for column, name in enumerate(('F_state', 'F_atom')):
52     ax = axes[0, column]
53     for j, scheme in enumerate(schemes):
54         for selected in (False, True):
55             ax.plot(results['D'], results[name][j, int(selected)],
56                     'o-' if selected else 'o--', label=f'{scheme}, selection={selected}')
57 ax.set(xlabel='D', ylabel=f'time mean {name}')
58 ax.legend(fontsize=8)
59 for j, scheme in enumerate(schemes):
60     ax = axes[j + 1, column]
61     image = ax.pcolormesh(results['photon_windows'], results['atom_windows'],
62                           results[f'{name}_heatmap'][j].T, shading='nearest', vmin=0, vmax=1)
63     fig.colorbar(image, ax=ax)
64     ax.set(xlabel='packet window / sigma_k', ylabel='resonance window / sigma_k',
65           title=f'{scheme}: {name}')
66 fig.tight_layout()
67 return fig
68
69
70 def main(argv=None):
71     p = parser(__doc__)
72     p.add_argument('--D-values', default='0.02,0.1,0.2')
73     p.add_argument('--photon-windows', default='0.5,1.5,3.0')
74     p.add_argument('--atom-windows', default='0.0,0.5,1.5')
75     args = p.parse_args(argv)
76     config = config_from_args(args)
77     from src.experiment import estimate_config
78     estimate_config(config)
79     estimate_config(replace(config, mode_selection=True))
80     results = run_mode_selection(config, [float(v) for v in args.D_values.split(',')],
81                                     photon_windows=[float(v) for v in args.photon_windows.split(',')],
82                                     atom_windows=[float(v) for v in args.atom_windows.split(',')], progress=args.
83                                     progress)
84     finish(plot_mode_selection(results), args, config, **results)
85
86 if __name__ == '__main__':
87     main()

```

Lines	Mathematical and implementation commentary
11–44	<code>run_mode_selection</code> : Use an unselected total-cap comparator at each coupling. Sweep schemes and both selection flags; return $(S, 2, Q)$ means/initial metrics. At one fixed D , scan windows to return (S, P, A) mean-only heatmaps.
47–67	<code>plot_mode_selection</code> : Plot selection on/off sweeps and window maps for both metrics. Transpose the stored (P, A) slice for plotting so the horizontal axis remains photon window and vertical axis atom window.
70–83	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

```

experiment/energy_profile.py
1 """Energy heatmaps and animation in mode and photon-number coordinates."""
2
3 import numpy as np
4
5 from ._common import config_from_args, finish, parser, run
6
7
8 def run_energy_profile(config, progress=False):
9     if not config.store_state:
10         raise ValueError('Energy time profiles require store_state=True')
11     experiment = run(config, progress)
12     k, modes, _, atom = experiment.compute_energy_profile_modes()
13     numbers, excitation = experiment.compute_energy_profile_excitations()
14     experiment.energy_profiles = {'time': experiment.times, 'k': k, 'E_modes': modes,
15                                  'E_atom': atom, 'photon_numbers': numbers,
16                                  'E_excitation': excitation, 'E_total': experiment.compute_energy()}
17     return experiment

```

```

18
19
20 def plot_energy_profile(experiment):
21     import matplotlib.pyplot as plt
22     data = experiment.energy_profiles
23     fig, axes = plt.subplots(1, 3, figsize=(13, 4))
24     for ax, x, values, xlabel in ((axes[0], data['k'], data['E_modes'], 'k'),
25                                   (axes[1], data['photon_numbers'], data['E_excitation'], 'photon number')):
26         image = ax.pcolormesh(x, data['time'], values, shading='nearest')
27         fig.colorbar(image, ax=ax)
28         ax.set(xlabel=xlabel, ylabel='t')
29     axes[2].plot(data['time'], data['E_total'], label='total')
30     axes[2].plot(data['time'], data['E_atom'], label='TLS')
31     axes[2].set(xlabel='t', ylabel='energy')
32     axes[2].legend()
33     fig.tight_layout()
34     return fig
35
36
37 def animate_energy_profile(experiment, path=None, fps=15, stride=1):
38     import matplotlib.pyplot as plt
39     from matplotlib.animation import FuncAnimation, PillowWriter
40     data = experiment.energy_profiles
41     fig, axes = plt.subplots(1, 2, figsize=(10, 4))
42     field, = axes[0].plot(data['k'], data['E_modes'][0], 'o-', label='field')
43     atom, = axes[0].plot([0], [data['E_atom'][0]], 's', label='TLS energy')
44     bars = axes[1].bar(data['photon_numbers'], data['E_excitation'][0])
45     for ax, values in ((axes[0], np.append(data['E_modes'], data['E_atom'])),
46                       (axes[1], data['E_excitation'])):
47         lower, upper = values.min(), values.max()
48         pad = max((upper - lower) * 0.1, 1e-6)
49         ax.set_ylim(lower - pad, upper + pad)
50         ax.set_ylabel('energy')
51     axes[0].set_xlabel('k')
52     axes[0].legend()
53     axes[1].set_xlabel('photon number')
54     def update(i):
55         field.set_ydata(data['E_modes'][i])
56         atom.set_ydata([data['E_atom'][i]])
57         for bar, value in zip(bars, data['E_excitation'][i]):
58             bar.set_height(value)
59         fig.suptitle(f't={data["time"][i]:.3f}')
60         return field, atom, *bars
61     animation = FuncAnimation(fig, update, frames=range(0, len(data['time']), stride),
62                              interval=1000 / fps, blit=False)
63
64     if path is not None:
65         from pathlib import Path
66         Path(path).parent.mkdir(parents=True, exist_ok=True)
67         animation.save(path, writer=PillowWriter(fps=fps))
68         plt.close(fig)
69     return animation
70
71 def main(argv=None):
72     p = parser(__doc__)
73     p.add_argument('--gif', type=str)
74     args = p.parse_args(argv)
75     config = config_from_args(args)
76     from src.experiment import estimate_config
77     estimate_config(config)
78     experiment = run_energy_profile(config, args.progress)
79     if args.gif:
80         animate_energy_profile(experiment, args.gif)
81     finish(plot_energy_profile(experiment), args, config, **experiment.energy_profiles)
82
83
84 if __name__ == '__main__':
85     main()

```

Lines	Mathematical and implementation commentary
8–17	<code>run_energy_profile</code> : Require history, propagate once, and attach mode/TLS, photon-number and total energies to the experiment. Their shapes and sum identities are documented above.
20–34	<code>plot_energy_profile</code> : Plot time-momentum and time-photon-total heatmaps plus total/TLS energy. Negative partition values are energies, not invalid probabilities.
37–68	<code>animate_energy_profile</code> : Reuse existing arrays for animation and optional GIF output, with stride and frame rate affecting display only. The marker at momentum zero represents separate TLS energy, not another oscillator.
71–81	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

```

experiment/full_cumulative_fidelity.py
1  """Final-state convergence between neighbouring total photon caps."""
2
3  from dataclasses import replace
4
5  import numpy as np
6
7  from src.fidelities import compare_states
8  from .common import config_from_args, finish, parser, run
9
10
11 def run_cap_convergence(config, caps=(1, 2, 3, 4), progress=False):
12     caps = list(caps)
13     if len(caps) < 2 or any(b <= a for a, b in zip(caps, caps[1:])):
14         raise ValueError('At least two strictly increasing caps are required')
15     runs = [run(replace(config, n_max=cap, truncation='full+totalcap', store_state=False), progress)
16             for cap in caps]
17     final, initial = [], []
18     for a, b in zip(runs, runs[1:]):
19         final.append(compare_states(a.result.final_state, a.basis, a.k_tab,
20                                   b.result.final_state, b.basis, b.k_tab))
21         initial.append(compare_states(a.state0, a.basis, a.k_tab, b.state0, b.basis, b.k_tab))
22     return {'caps': np.array(caps), 'runs': runs,
23           **{name: np.array([row[name] for row in final]) for name in ('F_state', 'F_atom')},
24           **{f'{name}_initial': np.array([row[name] for row in initial])
25             for name in ('F_state', 'F_atom')}}
26
27
28 def plot_cap_convergence(results):
29     import matplotlib.pyplot as plt
30     fig, axes = plt.subplots(1, 2, figsize=(10, 4))
31     for ax, name in zip(axes, ('F_state', 'F_atom')):
32         ax.plot(results['caps'][:-1], results[name], 'o-', label='final')
33         ax.plot(results['caps'][:-1], results[f'{name}_initial'], 'o--', label='initial')
34         ax.set(xlabel='lower cap of compared pair', ylabel=name, ylim=(0, 1.05))
35         ax.legend()
36     fig.tight_layout()
37     return fig
38
39
40 def main(argv=None):
41     p = parser(__doc__)
42     p.set_defaults(photon_state='coherent')
43     p.add_argument('--caps', default='1,2,3,4')
44     args = p.parse_args(argv)
45     config = config_from_args(args)
46     caps = [int(value) for value in args.caps.split(',')]
47     from src.experiment import estimate_config
48     estimate_config(replace(config, n_max=max(caps), truncation='full+totalcap', store_state=False))
49     results = run_cap_convergence(config, caps, args.progress)
50     finish(plot_cap_convergence(results), args, config,
51           **{key: value for key, value in results.items() if key != 'runs'})
52
53

```

```

54 if __name__ == '__main__':
55     main()

```

Lines	Mathematical and implementation commentary
11–25	run_cap_convergence: Require at least two strictly increasing caps; run total-cap final-only trajectories at each. Compare adjacent list entries initially and finally using physical embedding; retain all run objects.
28–37	plot_cap_convergence: Two metric panels distinguish initial and final fidelities. The horizontal value is the lower cap in each adjacent pair, not necessarily a comparison of N with $N + 1$.
40–51	main: Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

```

experiment/scattering.py
1 """Gaussian photon scattering, with the user's original dictionary API."""
2
3 import csv
4 from pathlib import Path
5
6 import numpy as np
7
8 from src.xp_config import ExperimentConfig
9 from .common import config_from_args, finish, parser, run
10
11
12 def run_scattering(param_photon, param_atom, param_timeevol, cutoffs=None,
13                   n_max=3, truncation='full+totalcap', RWA=False, store_state=True,
14                   store_results=False, progress=False, CTRL_M_EXPLICIT=False, M=None):
15     config = ExperimentConfig(param_photon, param_atom, param_timeevol, cutoffs,
16                              n_max, truncation, RWA, store_state,
17                              CTRL_M_EXPLICIT=CTRL_M_EXPLICIT, M=M)
18     experiment = run(config, progress)
19     if store_results:
20         path = Path(__file__).resolve().parents[1] / 'results' / 'scattering.csv'
21         path.parent.mkdir(parents=True, exist_ok=True)
22         with path.open('w', newline='') as handle:
23             writer = csv.writer(handle)
24             writer.writerow(experiment.observables)
25             writer.writerows(zip(*experiment.observables.values()))
26     return experiment
27
28
29 def plot_scattering(experiment):
30     import matplotlib.pyplot as plt
31     fig, axes = plt.subplots(1, 2, figsize=(11, 4))
32     times = experiment.observables['time']
33     x = np.linspace(-experiment.param_atom['L'] / 2, experiment.param_atom['L'] / 2, 500, endpoint=False)
34     for i in sorted({0, len(times) // 2, len(times) - 1}):
35         phi = experiment.one_photon_wavefunction(i, x)
36         axes[0].plot(x, abs(phi) ** 2, label=f't={times[i]:.2f}')
37         # With  $u \sim \exp(-ikx_{\text{tls}})$  and  $\phi \sim \exp(+ikx)$ , the spatial coupling is at  $-x_{\text{tls}}$ .
38         axes[0].axvline(-experiment.param_atom['x_tls'], color='black', linestyle=':',
39                        label='interaction coordinate ( $-x_{\text{tls}}$ )')
40         axes[0].set(xlabel='x', ylabel=r'$|\phi_{1,g}(x,t)|^2$')
41         for name in ('p_transmitted', 'p_reflected', 'p_zero_1g', 'p_excited', 'norm'):
42             axes[1].plot(times, experiment.observables[name], label=name)
43         axes[1].set(xlabel='t', ylabel='population')
44     for ax in axes:
45         ax.legend()
46     fig.tight_layout()
47     return fig
48
49
50 def main(argv=None):
51     p = parser(__doc__)
52     args = p.parse_args(argv)
53     config = config_from_args(args)

```

```

54 from src.experiment import estimate_config
55 estimate_config(config)
56 experiment = run_scattering(config.param_photon, config.param_atom, config.param_time_evol,
57                             config.cutoffs, config.n_max, config.truncation, config.RWA,
58                             progress=args.progress, CTRL_M_EXPLICIT=config.CTRL_M_EXPLICIT, M=config.M)
59 finish(plot_scattering(experiment), args, config, **experiment.observables)
60
61
62 if __name__ == '__main__':
63     main()

```

Lines	Mathematical and implementation commentary
12–26	<code>run_scattering</code> : Preserve the user dictionary API and append grid-control inputs. Run shared dynamics; optional CSV contains observable columns only and overwrites its fixed repository results path.
29–47	<code>plot_scattering</code> : Plot initial/middle/final $1g$ Fourier densities and directional/zero/TLS/norm populations. The interaction marker is $-x_{\text{tls}}$, derived in (47).
50–59	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

```

experiment/sweep_D_fidelity.py
1 """Time-mean state/TLS fidelities versus the physical coupling D."""
2
3 from dataclasses import replace
4
5 import numpy as np
6
7 from src.fidelities import fidelities_over_time
8 from .common import config_from_args, finish, parser, run
9
10
11 def run_coupling_sweep(config, D_values, schemes=('truncated',), reference='full+totalcap', progress=False):
12     D_values = np.asarray(D_values, dtype=float)
13     if D_values.ndim != 1 or len(D_values) == 0 or not np.isfinite(D_values).all():
14         raise ValueError('D_values must be a nonempty finite one-dimensional array')
15     schemes = tuple(schemes)
16     shape = (len(schemes), len(D_values))
17     out = {'D': D_values, 'schemes': schemes, 'reference': reference,
18           **{name: np.empty(shape) for name in
19              ('F_state', 'F_atom', 'F_state_initial', 'F_atom_initial')}}
20     for i, D in enumerate(D_values):
21         at_D = replace(config.with_D(float(D)), store_state=True)
22         ref = run(replace(at_D, truncation=reference), progress)
23         for j, scheme in enumerate(schemes):
24             candidate = ref if scheme == reference else run(replace(at_D, truncation=scheme), progress)
25             fidelities = fidelities_over_time(candidate, ref)
26             for name, values in fidelities.items():
27                 out[name][j, i] = np.mean(values)
28                 out[f'{name}_initial'][j, i] = values[0]
29     return out
30
31
32 def plot_coupling_sweep(results):
33     import matplotlib.pyplot as plt
34     fig, axes = plt.subplots(1, 2, figsize=(10, 4))
35     for ax, name in zip(axes, ('F_state', 'F_atom')):
36         for j, scheme in enumerate(results['schemes']):
37             ax.plot(results['D'], results[name][j], 'o-', label=scheme)
38             ax.plot(results['D'], results[f'{name}_initial'][j], '--', alpha=0.5)
39         ax.set(xlabel='D', ylabel=f'time mean {name}', ylim=(0, 1.05))
40         ax.legend()
41     fig.tight_layout()
42     return fig
43
44
45 def main(argv=None):

```

```

46 p = parser(__doc__)
47 p.add_argument('--D-values', default='0.02,0.1,0.2')
48 p.add_argument('--schemes', default='truncated')
49 p.add_argument('--reference', choices=('full', 'full+totalcap', 'truncated'), default='full+totalcap')
50 args = p.parse_args(argv)
51 config = config_from_args(args)
52 from src.experiment import estimate_config
53 for scheme in set(args.schemes.split(',')) | {args.reference}:
54     estimate_config(replace(config, truncation=scheme))
55 results = run_coupling_sweep(config, [float(v) for v in args.D_values.split(',')],
56                                args.schemes.split(','), args.reference, args.progress)
57 finish(plot_coupling_sweep(results), args, config, **results)
58
59
60 if __name__ == '__main__':
61     main()

```

Lines	Mathematical and implementation commentary
11–29	<code>run_coupling_sweep</code> : Validate a finite nonempty one-dimensional D array and force histories. At each D , propagate the chosen comparator and each candidate, reusing it for an identical scheme. Return (S, Q) sample means and initial metrics, (128).
32–42	<code>plot_coupling_sweep</code> : Plot sample means versus physical D and dashed initial values, separately for both metrics. The unsuffixed result is neither final fidelity nor a time-integral quadrature.
45–57	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

3 Comparison with the student implementation and validation

3.1 Exact convention conversions and finite-model changes

Coupling, field phase and state conversion. An exact comparison first needs identical physical modes, bases and dispersion. Call the historical parameter named $\mathbf{g}$ γ_U ; its annihilation coefficient was $\gamma_U \sqrt{|k_m|} e^{-ik_m x_a}$. For sqrt and equal position parameters, matching magnitudes requires

$$D = \sqrt{L} \gamma_U. \quad (139)$$

The box factor means equal numerical old/new couplings need not agree. The factor i is a field phase convention implemented by

$$U = e^{-i\pi \mathcal{N}_f/2}, \quad U a_m U^\dagger = i a_m, \quad U a_m^\dagger U^\dagger = -i a_m^\dagger, \quad H_D = U H_U U^\dagger. \quad (140)$$

Since $U |n; s\rangle = (-i)^\nu |n; s\rangle$, applying a ladder before and after the diagonal phase proves the rules. All occupation projectors commute with U , so the equality survives truncation. Matched trajectories also require

$$|\psi_D(0)\rangle = U |\psi_U(0)\rangle, \quad |\psi_D(t)\rangle = U |\psi_U(t)\rangle. \quad (141)$$

Fixed-photon-number preparation changes only by a global phase. Coherent preparation instead requires

$$\alpha_D = -i\alpha_U, \quad (142)$$

including after projection. Keeping the same complex α while changing the Hamiltonian phase is not a matched-state conversion. Correctly related trajectories have equal TLS reductions and number populations; phase-sensitive field observables must be transformed too. Fidelities are invariant

when the same U acts on both compared states. Transforming only one member can produce a convention-dependent overlap.

A displaced TLS additionally satisfies

$$H(x) = U_x H(0) U_x^\dagger, \quad U_x = \exp\left(ix \sum_m k_m n_m\right), \quad U_x a_m U_x^\dagger = e^{-ik_m x} a_m. \quad (143)$$

This checks both transition phases simultaneously. It does not alter the Fourier-coordinate interpretation in equation (47).

Packet-width and selection conversion. The historical amplitude was $\exp[-\sigma_U^2(k - k_0)^2/2]$. Equating exponents with equation (15) gives

$$\sigma_k = \frac{1}{\sqrt{2}\sigma_U}, \quad x_0 = x_{\text{photon},U}, \quad k_0 = k_{\text{photon},U}. \quad (144)$$

The old width multiplies momentum differences; the new width is a momentum standard-deviation parameter. Increasing old σ_U narrows the momentum packet. Normalization on identical modes then gives the same coefficients.

Historical selection radii were $w_U \sigma_U$, whereas new radii are $w_D \sigma_k$. Equal numerical radii additionally require

$$w_D = \frac{w_U \sigma_U}{\sigma_k} = \sqrt{2} w_U \sigma_U^2. \quad (145)$$

This is a conversion in the historical numerical units. Envelope conversion alone does not match a selected calculation: the window radii and nearest-mode fallback must also be checked. Current factors mean multiples of the actual momentum width, rather than automatically reproducing old factor values.

Implicit UV regulation and the changed grid. The historical folded grid used L and requested M_U . For even $M_U = 2J$, its retained indices after zero deletion were

$$\{-J+1, \dots, -1, 1, \dots, J\}, \quad |\mathcal{K}_U| = M_U - 1, \quad k_{\max} = J\Delta k = \frac{\pi M_U}{L}. \quad (146)$$

The positive endpoint was unpaired. Full bandwidth is approximately $2\pi M_U/L$ and radial UV approximately $\pi M_U/L$: these are different uses of “cutoff”. Current UV means largest retained $|k|$. At $M_U = 16, L = 20$, old indices were $-7, \dots, -1, 1, \dots, 8$. New explicit $M = 15$ gives $-7, \dots, 0, \dots, 7$; $M = 17$ gives $-8, \dots, 0, \dots, 8$. Neither reproduces the old set. The symmetric cutoff branch also cannot reproduce its unpaired endpoint.

This is an intentional change of finite model, not a stylistic adjustment or a coupling redefinition. The new UV values at $L = 20$ are $7\pi/10$ and $4\pi/5$ respectively. If one wants a fixed physical UV while increasing L and refining momentum spacing, cutoff control states that intention explicitly and lets the count follow. Positive IR excludes zero only by an explicit band choice, not a hard-coded special case.

Parameter migration and compatibility boundaries. The numerical conversion rules can be located directly in the current inputs:

Historical input	Current input	Meaning and required check
<code>length</code>	Atomic <code>L</code>	Same physical length in natural units; the new momentum list must still be checked explicitly.
<code>modes</code>	Config <code>M</code> or cutoffs	Odd actual count under explicit control; it is not the historical pre-deletion counter.
<code>g</code>	Atomic <code>D</code>	$D = \sqrt{L} \gamma_U$ for sqrt and matched units; coherent phase also requires (142).
<code>w_atom, x_atom</code>	Atomic <code>omega_0, x_tls</code>	Same frequency and Hamiltonian phase parameter. Do not inherit the old nonzero default position when intending a TLS at zero.
<code>k_photon, x_photon</code>	Photon <code>k_0, x_0</code>	Same signed packet centre and phase parameter at matched modes.
<code>sigma_photon</code>	Photon <code>sigma_k</code>	Reciprocal-width conversion (144), not a direct rename.
<code>state class</code>	Photon <code>state</code> string	<code>number</code> or <code>coherent</code> ; explicit <code>n</code> or <code>alpha</code> in the photon dictionary.
<code>atom_state</code>	Atomic <code>initial_state</code>	Current preparation interface accepts <code>g, e, +, -</code> labels. The former arbitrary two-component sequence is not an accepted label in this interface.
<code>excitation_cap</code>	Config <code>n_max</code>	Cap meaning depends on the basis: per-mode in full, total in total-cap, ray occupation in truncated.
<code>t, dt</code>	Time <code>T, dt</code>	Same duration and nominal sampling intention, but a changed output-time construction.
<code>hbar, c</code>	Fixed natural units	The current API assumes both equal one; it does not expose arbitrary dimensional constants.

The old special single-mode branch placed a mode at the packet momentum. Current explicit $M = 1$ instead means the zero mode, as specified by the symmetric-grid rule. Legacy classes such as `Config` and the separate Hamiltonian/state subclasses are replaced by the shared configuration and engine. The executable Uri paths are maintained; the historical internal class API and argument syntax are not preserved as aliases. The user's dictionary-based scattering interface is retained.

Sampling changes also have a concrete consequence. At $T = 4$ and $dt=0.05$, the old construction requested 80 outputs with spacing $4/79$, whereas the current construction requests 81 outputs with spacing 0.05. A mean over those samples is a different discrete statistic even if both solvers approximate the same continuous trajectory. In the old construction, `dt` was used to choose a count rather than enforced as an output spacing.

Comparative summary and corrected overlap.

Item	Student implementation	Patch and consequence
Grid	Requested count reduced by zero deletion; unpaired end-point possible.	Exact odd count with zero or an explicit radial band; reports resulting count and effective cutoffs.

Item	Student implementation	Patch and consequence
Coupling	Historical parameter, real coefficient at zero position.	$iDf_m/\sqrt{L}$ with signed phase. Exact conversion at fixed grid is (139)–(141). Uri notebooks use <code>sqrt</code> .
Packet	Multiplicative width in exponent.	Momentum width in denominator, conversion (144); all selected momentum signs enter preparation.
Assembly	Search among pairs of states.	Dictionary targets and bosonic transitions, $O(Md)$ assembly; shared entries fixed by (33).
Indexing	Separate classes and scheme-specific indices.	Occupation tuples, alternating TLS entries and comparisons of physical configurations.
Cross-basis overlap	Different basis types compared through one-occupied-mode configurations.	All shared occupations in implicit full, physical momentum alignment, and vacuum embedding of absent modes.
Metrics	Global/atomic metrics plus a ratio used as a field proxy.	Only squared state/atomic fidelities; initial differences separately visible.
Times	<code>linspace(0,T,int(T/dt))</code> and output renormalization.	Nominal <code>dt</code> , exact final time, raw norms and explicit tolerances. Sample means may change even with similar trajectories.
Control	Heterogeneous scripts and inputs.	Callable modules, six notebooks, common CLI and historical path delegates. Former <code>codex/</code> migrated and removed.
Energy	Mode/photon-number decompositions.	Preserved sum rules with interaction terms assigned using shared sparse transition data.

A minimal counterexample to the old cross-type overlap is $|1, 1; g\rangle$ in full and total-cap with $N = 2$. The physical overlap is one; a sum over one occupied mode gives zero. The corrected pair returns both fidelities equal to one. This fixes a comparison, not a propagated trajectory.

At identical grids, number preparations are identical in every basis that contains them, and remain identical for $D = 0$. Coherent projections can have equation (27) below one. Since occupation projectors commute with H_0 , their mutual overlap remains at that initial value during free evolution on the shared grid. This diagnoses preparation differences without mistaking them for interaction-induced error.

Reproducing old figures requires the exact mode list and cap/basis, coupling magnitude and phase, envelope and coherent phase, selection radii, output sampling and statistic. Parameter labels alone cannot establish reproduction when the finite grids or initial projections differ.

3.2 Mathematical verification and limits of the conclusions

Independent finite-dimensional oracles. Tests construct small oscillator/TLS tensor operators independently of the transition loops, select rows/columns for each occupation basis and check

$$H_{\text{code}} = P_{\mathcal{B}} H_{\text{tensor}} P_{\mathcal{B}}, \quad H_{\text{code}} = H_{\text{code}}^{\dagger}. \quad (147)$$

Checks cover three bases, two profiles, RWA on/off and zero/nonzero positions. They also test parity, RWA excitation conservation and the simultaneous annihilation/creation phase identity (143).

An analytic check takes $L = 2\pi$, modes $\{-1, 0, 1\}$, sqrt, $\omega_0 = 1$, RWA, $N \geq 1$ and input $|0; e\rangle$. At zero TLS position,

$$|B\rangle = (|1_-; g\rangle + |1_+; g\rangle)/\sqrt{2}, \quad |\text{dark}\rangle = (|1_-; g\rangle - |1_+; g\rangle)/\sqrt{2}. \quad (148)$$

The dark and zero-mode states decouple. In $(|0; e\rangle, |B\rangle)$,

$$H = I + \begin{pmatrix} 0 & iG \\ -iG & 0 \end{pmatrix}, \quad G = \frac{\sqrt{2}D}{\sqrt{L}}, \quad P_e(t) = \cos^2(Gt). \quad (149)$$

This checks the two-mode factor, box normalization and evolution in all bases. Independent $e^{-iHt}\psi_0$ comparisons check BDF and final-only storage.

Fidelities are tested against explicit small common-space embeddings with reordered grids, selected subsets, caps and shared multimode configurations. Atomic fidelity is checked against equation (124). The (1,1) case protects the correction; orthogonal field directions with identical TLS protect the distinction between metrics. Population sums and energy partitions are checked against direct norms and expectations. Gaussian, vacuum, coherent and zero-mode preparations are tested separately from evolution.

Integration tests exercise five historical script paths, exports and all six callable experiments. The six notebooks were validated and executed cell by cell at their defaults, leaving saved outputs empty. The completed migration passed 99 tests. These are migration validation results; a documentation rebuild does not replace rerunning tests after future code changes. In an installed development environment, use:

```
1 python -m pytest tests -q
```

Documentation compiles from its sources and regenerates line comments from current files. Derivations added here explain the current model; they do not introduce additional solver features.

Integration error versus projection error. Norm and energy conservation are necessary checks, but two different unitary evolutions can preserve both. Tighter tolerances or an independent exponential reference on small systems test integration error more directly. For an interpolated numerical trajectory with $r(t) = i\dot{v}_{\text{num}} - H_{\mathcal{B}}v_{\text{num}}$, variation of constants gives

$$\|v_{\text{num}}(t) - v(t)\| \leq \|v_{\text{num}}(0) - v(0)\| + \int_0^t \|r(s)\| ds. \quad (150)$$

The engine reports sampled diagnostics, not an evaluated residual integral; this equation specifies what a residual-based certificate would require.

For projection, take a larger finite space with Hamiltonian H , smaller projector P , and $Q = I - P$. Let ψ solve the larger evolution and ψ_P solve PHP . For $e = P\psi - \psi_P$,

$$i\dot{e} = PHPe + PHQ\psi, \quad \|e(t)\| \leq \|e(0)\| + \int_0^t \|PHQ\psi(s)\| ds. \quad (151)$$

The driving term depends on omitted dynamics, not just final boundary population. Normalized coherent projections can give $e(0) \neq 0$. Cap comparisons are useful, but a small final boundary population alone is not a rigorous dynamical error certificate. The modules do not evaluate this bound.

Regulator convergence and finite-box scattering. The checks establish finite-model consistency, not physical continuum convergence. A study still needs separate variations of

$$N \rightarrow N', \quad k_{\text{UV}} \rightarrow k'_{\text{UV}}, \quad k_{\text{IR}} \rightarrow k'_{\text{IR}}, \quad L \rightarrow L', \quad (\text{rtol}, \text{atol}) \rightarrow (\text{rtol}', \text{atol}'). \quad (152)$$

Truncated omits multimode configurations at every N ; its large-cap limit is not full Fock space. Conversely, RWA number preparations can occupy a small invariant excitation sector in which larger caps make no difference. That agreement does not test arbitrary high-photon dynamics.

Changing L at fixed explicit count changes spacing, UV, resonant-mode availability and $D/\sqrt{L}$ together. Holding physical D and cutoffs fixed while changing L is a different discretization study. For a finite smooth band the associated Riemann sum is

$$\frac{1}{L} \sum_{k_m \in \mathcal{K}} f_m^2 \longrightarrow \frac{1}{2\pi} \int_{k_{\text{IR}} \leq |k| \leq k_{\text{UV}}} f(k)^2 dk. \quad (153)$$

This explains why per-mode $L^{-1/2}$ normalization and a growing mode count belong together. It does not prove convergence as UV increases without bound.

Free phases $e^{-i|j|\Delta kt}$ recur after $t = L$. Interactions can change exact recurrence times, but a periodic packet can return to the interaction region after circulating around the box. Scattering observations should separate incoming/outgoing packets before such returns and be checked in larger boxes. Increasing T without this check can turn single-passage scattering into repeated interaction. Initial negative/zero components and non-RWA multiphoton sectors must also be accounted for before assigning asymptotic coefficients.

Historical plots become numerical references only after their actual grids, preparations, coupling conversions and statistics have been reconstructed. The equations, visible notebook inputs and configuration metadata make these comparisons reproducible without treating a changed finite model as a code-style refinement.